Synaptic Labs'

xSPI-MC Memory Controller *(xSPI-MC-T001)* User Manual

SLL's unified xSPI Multiple Bus Memory Controller IP supports the widest range of JEDEC xSPI and xSPI-like NOR Flash and PSRAM memories (JEDEC xSPI Profile 1.0 and 2.0, HyperBus 1.0, 2.0 and 3.0, OctaBus and Xccela Bus) that are available now from many memory vendors.

SLL is an Intel Partner Alliance member

JEDEC xSPI and xSPI-like memories offer good performance with lower hardware and power costs. Memory device variants offer up to 512 Mbit PSRAM, up to 2 Gigabit NOR Flash, up to 250 MHz DDR clock speeds, with x4, x8 and x16 data path widths, and a wide range of package options including 4mm x 4mm BGA49 and tiny WLCSP footprints. Some PRSAM devices are now also available with internal ECC.

SLL's official partners include many of the leading global memory vendors. SLL works closely with our partners to ensure SLL's memory controllers are physically qualified with their memory device variants, significantly reducing your project's risks.

SLL's small xSPI MBMC v3.3.x IP enables you to easily evaluate, select and adopt the benefits of the latest xSPI-style memories in your projects and products.

SLL provides world class pre-sales and post-sales technical support for all the major memory vendors and FPGA vendors, helping you navigate the rapidly evolving market, on the platform of your choice.

Table of Contents

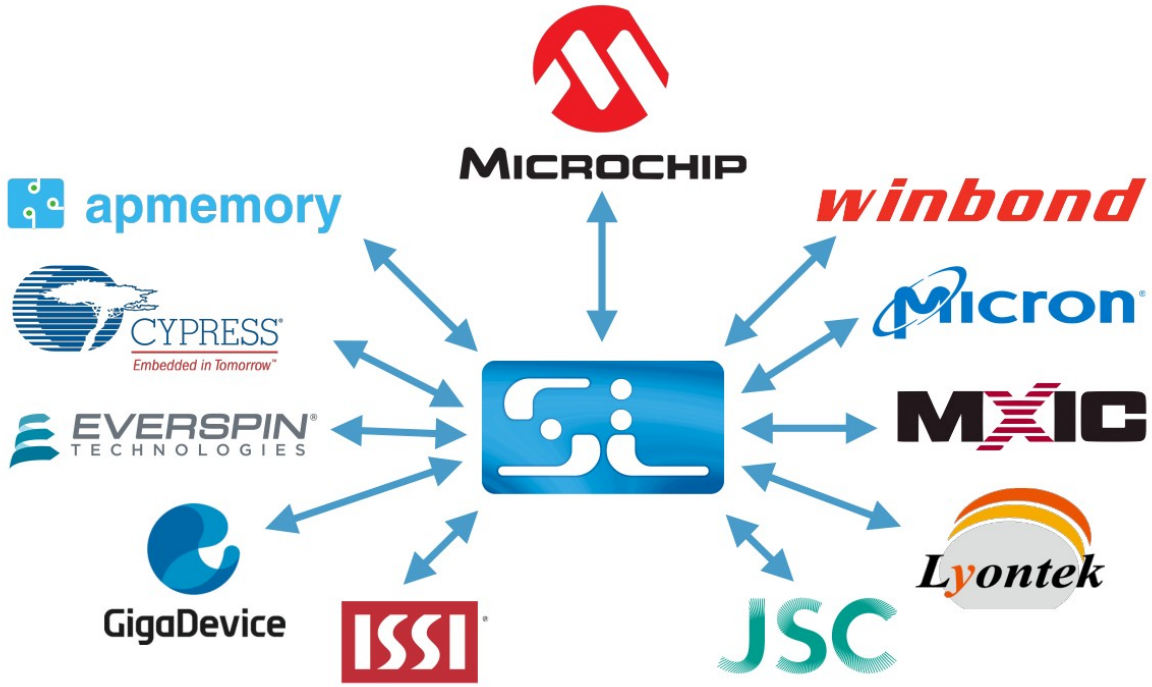
Synaptic....1
Introduction....4
1.0 Set-Up Requirements....9
2.0 Finding SLL xSPI Memory Controller IP in the Qsys IP Catalog....10
2.1 Instantiating SLL xSPI Memory Controller IP....11
2.2 Configuring SLL xSPI-MC IP in the Master User Interface Tab....12
2.2.1 PSRAM/Flash Channel Configuration Section....13
2.1 Memory Channel 0/1 Configuration....16
2.2.2 Memory IO Configuration....18
2.2.3 Memory Powerup Configuration....20
2.2.4 RWDS-DQ Skew Configuration....21
2.2.5 Avalon Target interface Configuration....21
2.3 Configuring SLL xSPI-MC IP in the Clock and PLL Configuration Tab....22
2.3.1 External PLL mode....22
....22
2.3.1.1 For Single Clock Operation....23
2.3.1.2 For Dual Clock Operation....23
2.4 Configuring SLL xSPI-MC IP in the IAVS0 Configuration Tab....24
2.4.1 Configuring Ingress Avalon port slave parameters....24
2.4.2 Configuring the burst count on the Ingress Avalon port slave....25
2.4.3 Configuring the Ingress Avalon port slave return path....26
2.5 Configuring SLL xSPI-MC IP in the Device 0/1 Tab....27
3.0 Connecting SLL xSPI Memory Controller to the NIOS II/f embedded processor....28
3.1 SLL xSPI Memory Controller with external PLL Configuration....30
3.1.1 Option A – Same Clock for the PSRAM/Flash memory channel an Avalon-MM channel....31
3.1.2 Clocking (PLL) Wizard Configuration....31
3.1.3 SLL xSPI-MC Configuration....31
3.1.4 SLL xSPI-MC wiring....32
3.2 Option B – Different Clocks for the PSRAM/Flash memory channel and Avalon-MM channel .
....33
3.2.1 Clocking Wizard Configuration....33
3.2.2 SLL MBMC Configuration....33
3.2.3 SLL xSPI-MC wiring....34
4.0 AVMM Limitations....35
4.1 Device Related Limitations....36
5.0 Signal Description....39
5.1 Memory Bus Signal Interface....39
5.1 Memory Clock Signal Interface....40
5.2 AVMM Clock/reset interface....41
5.3 AVMM Interface....41
5.4 AVMM register Control Interface....42
6.0 APB Register Control Port....43
6.1 Register MAP....45
6.2 Register Definitions....51
6.3 Accessing PsuedoRAM Registers....82
6.4 Accessing HyperFlash Memory using the Control Port....83
6.5 Accessing xSPI Flash Memory using the Control Port....84
6.6 xSPI Profile 1.0 Transaction : Format 1.A....85

6.8 xSPI Profile 1.0 Transaction : Format 1.B....87
6.9 xSPI Profile 1.0 Transaction : Format 1.C....88
6.10 xSPI Profile 1.0 Transaction : Format 1.D....89
6.11 SLL Low Level drivers....90
6.12 SLL Initialisation drivers....96
6.13 SLL Flash drivers....97
7.0 AXI4 Memory Address Mapping....98
8.0 xSPI Controller Core Operation....101
8.1 AXI Memory Response Error (*for AXI-inerfaces only*)....101
9.0 Core Reset....103
9.1 Global Asynchronous Reset....103
9.2 Software Core Reset *(not implemented)*....103
9.3 Software Device Reset....103
9.3 Device Sleep/Power Down *(to be further defined)*....104
10.0 Powering Up *(generic mode only)*....105
11.0 CycloneV Support....113
Additional QSF constraint....113
12.0 Static Timing Consideration....114
12.1 Data Input Timing Constraint....114
12.2 Timing Constraints....114
12.3 Pin Clustering....114
13.0 Using 3V PSRAM and HyperFlash devices....116
13.1 Selecting the correct operating frequency in Qsys....116
13.2 Selecting the correct voltage in Quartus....116
13.3 Connecting the PSRAM signals to the FPGA I/O....117

Introduction

SLL xSPI Memory Controller (xSPI-MC) supports the following Octal DDR PseudoRAMs and Flash from the following memory vendors :

- Infineon/Cypress/ISSI (HyperRAM , HyperFlash, SemperFlash, xSPI Flash, xSPI PRAM)
- Winbond (HyperRAM)
- AP Memory (OctaRAM and Xccela RAM)
- JSC (OctaRAM)
- LyonTek (Xccela RAM)
- Micron (xSPI Flash)
- GigaDevice (xSPI Flash)
- Macronix (MX25 Flash series)
- Everspin (EMLX series)



Please note that SLL xSPI-MC IP only supports flash and psram memory devices with the following configuration mode:

- DDR mode with Data Strobe (DS)

SLL xSPI-MC IP has a feature that enables the user to switch the memory device from SPI (X1) mode to DDR with Data Strobe mode (this can be used when the memory device start in SPI mode after reset).
The pseudoRAM / Flash memories operates in a Double Data Rate (DDR) mode. They have an 8-bit Serial Interface and are used in all applications such as automotive semiconductor market, Industrial Ethernet, AI, Wearable device market and IoT market. They support a maximum operating frequency of 200 MHz Speed and a maximum bandiwdth of 400 Mbits/s.
Further more, when compared to traditional DDR memories, the PseudoRAM memories are considered as low power alternatives.

Synaptic Labs' xSPI Memory Controller IP can be configured with an AXI4 or Avalon-MM slave interface.

Please note that not the FULL AXI4 protocol is implemented. These limitations are documented further below.) It supports burst access (up to 256 words).

The Avalon-MM slave interface supports burst mode access (up to 256 words). It can be configured in single access or burst mode access. Further more, burst mode access can be configured with either Avalon burst wrap support or Avalon burst-on-burst-boundaries support.

All external I/O pads to the memory signals are generated from within SLL xSPI-MC IP. The user does not need to manually instantiate the I/O pads in the design. The only exceptions are Microchip Polarfire, Intel's C10GX, Stratix 10 and Arria 10 FPGA targets.

Well documented example design projects and tutorials are provided upon request.

Note: Synaptic Labs' xSPI Memory Controller IP does NOT support DCARS functionality (Hyper-RAM PSC mode).

- DCARS is a very specific capability requested by a very specific customer / chipset partner and is only supported by Cypress devices.
- ISSI devices do not have DCARS support.
- Cypress do not recommend the implementation of DCARS functionality .
- DCARS has a maximum frequency of 133 MHz.

SLL is targeting the following 23 memory device families in xSPI-MC IP :

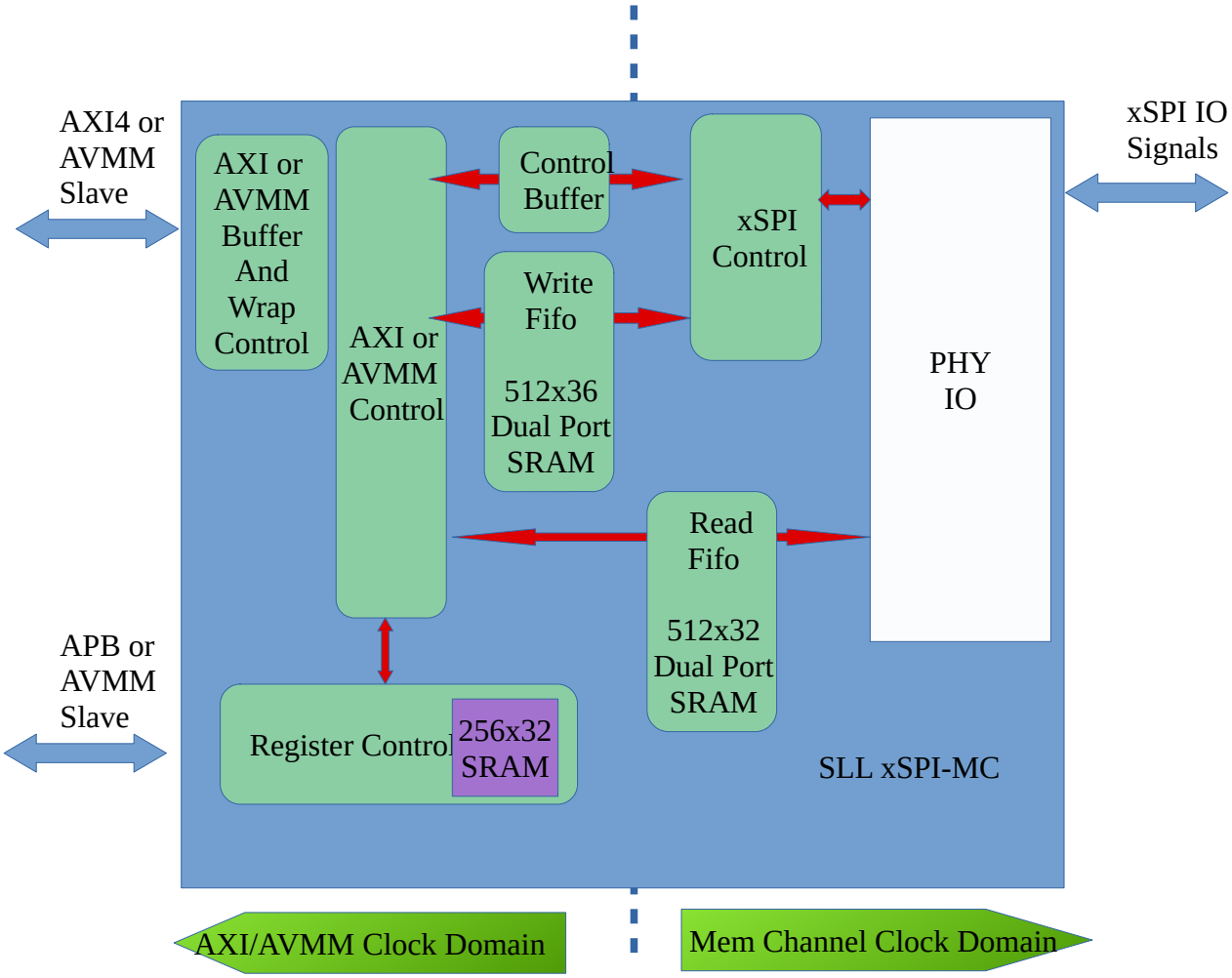
	Memory Vendor	1.8V Device Family	Device Type(s)	DQ (bits)	Device Speed (MHz)	Data Rate
1	AP Memory	Xccela PSRAM	PSRAM	8	200	DDR
2	AP Memory	OctaRAM	PSRAM	8	200	DDR
3	Cypress	HyperRAM 1.0	PSRAM	8	166	DDR
4	Cypress	HyperFlash 1.0	NOR Flash	8	166	DDR
5	Cypress	HyperMCP 1.0	PSRAM + NOR Flash	8	166	DDR
6	Cypress	HyperRAM 2.0	PSRAM	8	200	DDR
7	Cypress	Semper Flash with HyperBus Interface	NOR Flash	8	200	DDR
8	Cypress	HyperMCP 2.0	PSRAM + NOR Flash	8	200	DDR
9	Cypress	Semper Flash with Octal Interface (xSPI Profile 1.0)	NOR Flash	8	200	DDR
10	Everspin	STT-MRAM EMLX	MRAM	8	200	DDR
11	ISSI	HyperFlash 1.0	NOR Flash	8	166	DDR
12	ISSI	Octal Flash (xSPI Profile 1.0)	NOR Flash	8	200	DDR
13	ISSI	HyperRAM 1.0	PSRAM	8	166	DDR
14	ISSI	HyperRAM 1.0 with ECC	PSRAM	8	166	DDR
15	ISSI	OctalRAM	PSRAM	8	200	DDR
16	ISSI	OctalRAM with ECC	PSRAM	8	200	DDR
17	GigaDevice	GD25LX256E	NOR Flash	8	200	DDR
18	JSC	OctaRAM	PSRAM	8	200	DDR
19	Lyontek	Xccela PSRAM	PSRAM	8	200	DDR
20	Macronix	OctaFlash	NOR Flash	8	200	DDR
21	Micron	Xccela Flash	NOR Flash	8	200	DDR
22	Winbond	HyperRAM 1.0	PSRAM	8	166	DDR
23	Winbond	HyperRAM 2.0	PSRAM	8	200	DDR

PSRAM memory support is implemented predominantly in hardware. Certain PSRAM operations, such as entering and exiting low power mode, can be controlled using software via the control port of the xSPI-MC IP.

NOR Flash memory support is implemented as a combination of hardware and software device drivers:

- The data port of the xSPI-MC IP core in hardware is designed to permit:
- Fast read/write operations to PSRAM
- Fast read operations to NOR Flash
- The control port of the xSPI-MC IP core in hardware is designed to permit:
- All xSPI Profile 1.0 command types (1.A, 1.B, 1.C, 1.D), and
- The vast majority of the xSPI Profile 1.0 command type variations
- The control port of the xSPI-MC IP core in hardware will support:
    - All the command type variations employed in the above 23 memory device families.
    - Software drivers for NOR Flash memory devices can therefore be expanded over time (by SLL, Microchip, orthe customer) to access all memory device specific operations as required.

SLL xSPI-MC Block Diagram



Memory Bandwidth

The memory bandwidth of SLL xSPI-MC controller is close to the wire speed bandwidth of the memory device. The bandwidth depends on the memory channel frequency and the burst length of the memory transaction.

For a x8 memory channel running at 200 MHz DDR, and considering an initial access latency of 8, the effective memory bandwidth increase as the burst length increases:

Burst Length in Bytes	MHz	Initial Access Latency	Effective memory bandwidth in Megabytes/s	Wirespeed bandwidth in Megabytes/s	Effective memory bandwidth efficiency %
..	...	...	...	...	...
32	200	8	173.0	400.0	43.2
64	200	8	241.5	400.0	60.4
128	200	8	301.2	400.0	75.3
256	200	8	343.6	400.0	85.9
512	200	8	369.7	400.0	92.4
1024	200	8	384.2	400.0	96.1

For large burst lengths, the effective memory bandwidth is around 95% of the wire speed bandwidth and can be calculated as follows:

- Memory bandwidth in Mbytes/s = 0.95 * 2 * memory channel frequency

1.0 Set-Up Requirements

Step 1: Obtain core materials

1. Download and install Intel Quartus Standard (2022.1) on your PC, please ensure that your PC meets the required minimum specification.

Step 2: License Setup

1. Next you need to register for Synaptic Labs' xSPI-MC Memory Controller IP (info@synaptic-labs.com).

Step 3: Install AVMM xSPI-MC Vivado Component into the project IP Folder

1. In most tutorials we assume that SLL xSPI Memory Controller(xSPI-MC) will be located in the Project directory.
2. Copy SLL AXI xSPI-MC IP directory to the Quartus project directory/ip and ensure it is available in the IP Catalog.

Step 4: Extending maximum path length on Windows (optional)

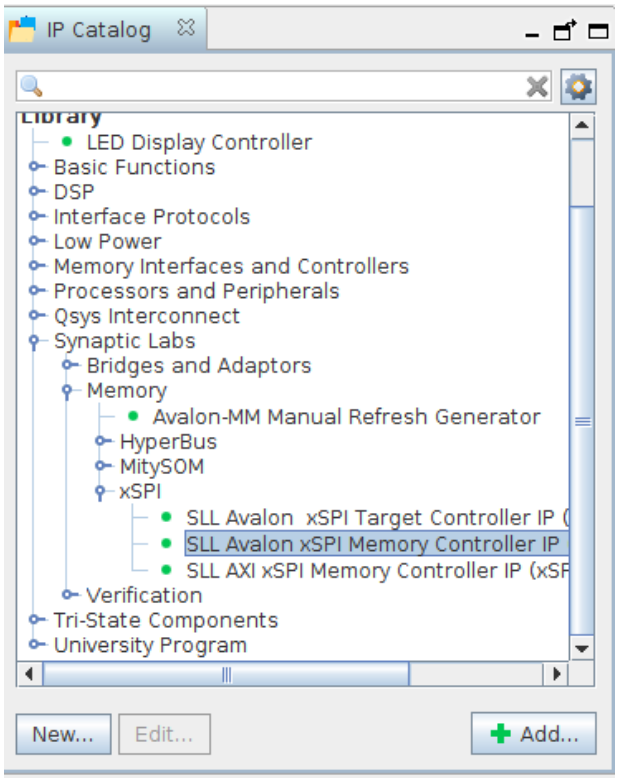
1. We suggest that you keep the maximum folder lengths as short as possible. There are problems if the path to the project / workspace is very long
2. On windows, should you wish to increase the maximum path from 260 characters to 32k characters, please folloe the guidance on how to increase the maximum path length:

   https://learn.microsoft.com/en-us/windows/win32/fileio/maximum-file-path-limitation?tabs=registry

2.0 Finding SLL xSPI Memory Controller IP in the Qsys IP Catalog

Within Qsys, SLL xSPI Memory Controller IP can be found in the Qsys IP Catalog panel under:

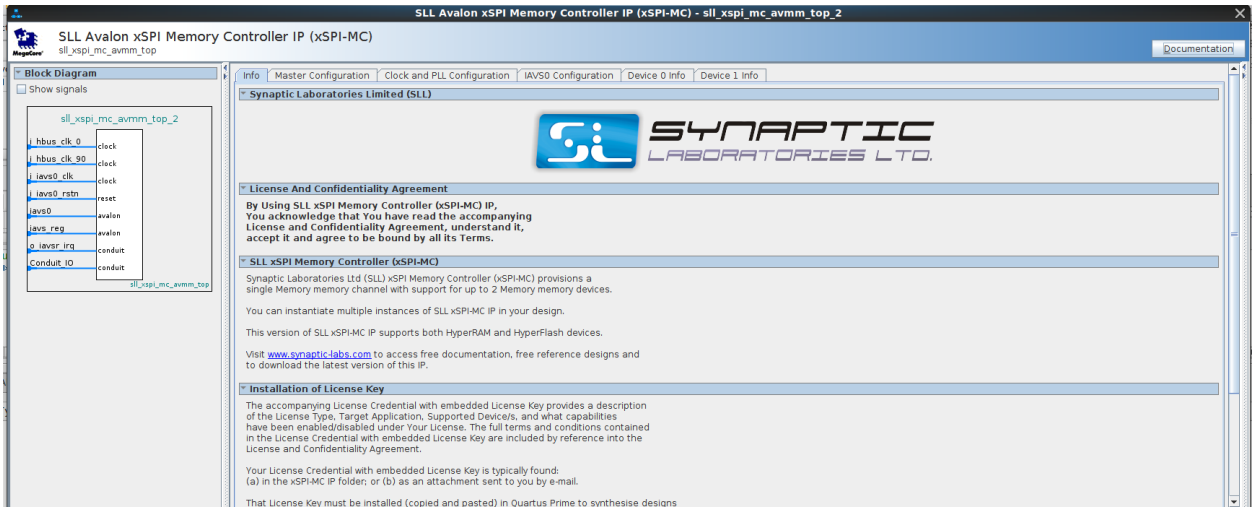
+ Synaptic Labs
 + Memory
  + xSPI
   + SLL Avalon xSPI Memory Controller



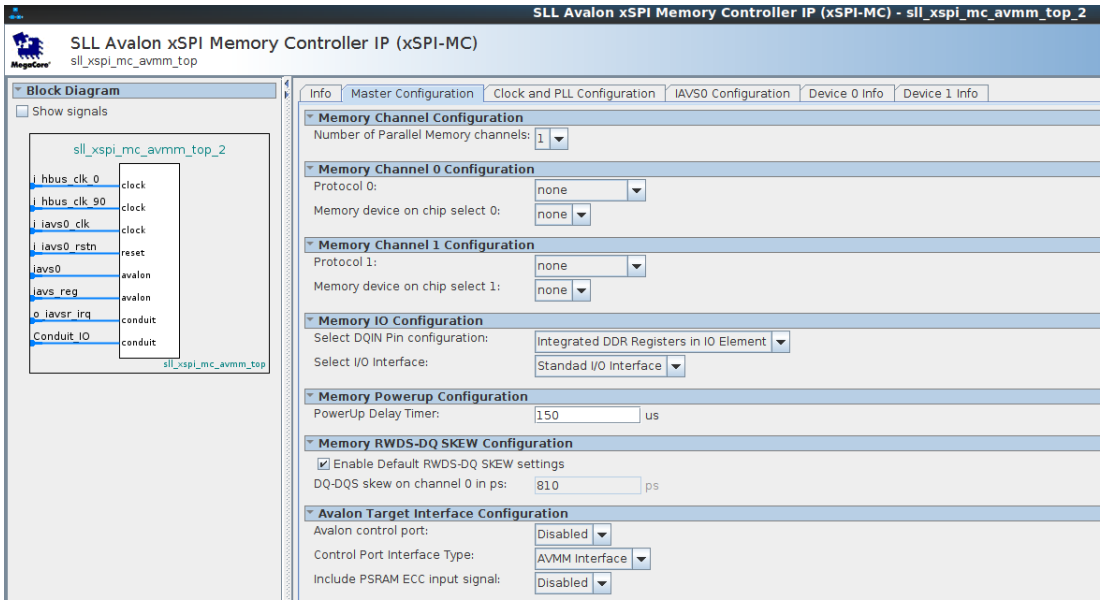
2.1 Instantiating SLL xSPI Memory Controller IP

Double click on Synaptic-Labs xSPI Memory Controller IP to instantiate an instance of this IP in your Qsys project.

The following IP configuration options will appear on your screen:



2.2 Configuring SLL xSPI-MC IP in the Master User Interface Tab



2.2.1 PSRAM/Flash Channel Configuration Section

In this section, the user selects the memory subsystem based on the development board.

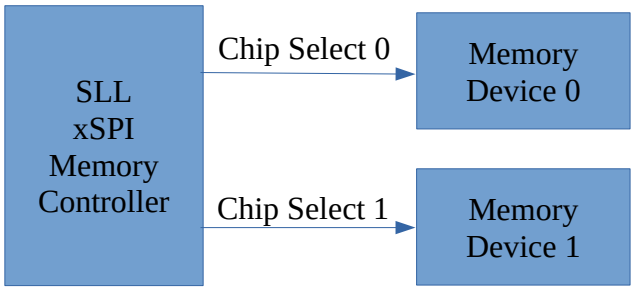
The following is a short description of SLL xSPI-MC IP parameters.

Memory Channel 0 Configuration contains a subset of configurable fields for the memory device connected on Chip Select 0.

If no device is present select **None** in the **Memory Protocol 0 Name**.

Memory Channel 1 Configuration contains a subset of configurable fields for the memory device connected on Chip Select 1.

If no device is present select **None** in the **Memory Protocol 1 Name**.



There are two options to configure the memory device selection mode in SLL xSPI-MBMC IP .

1) Fixed/Static Mode implementation

In this mode, the user configures the controller for the exact memory device connected on the board. After a power up delay, SLL xSPI-MBMC will automatically initialise the Flash or PSRAM configuration registers. The user does NOT need to program the memory controller in software. This implementation results in a much lower resource utilization in FPGA.

As expected, if the user decides to use a different memory, SLL xSPI-MBMC GUI configuration needs to be updated accordingly and the Quartus project re-compiled.

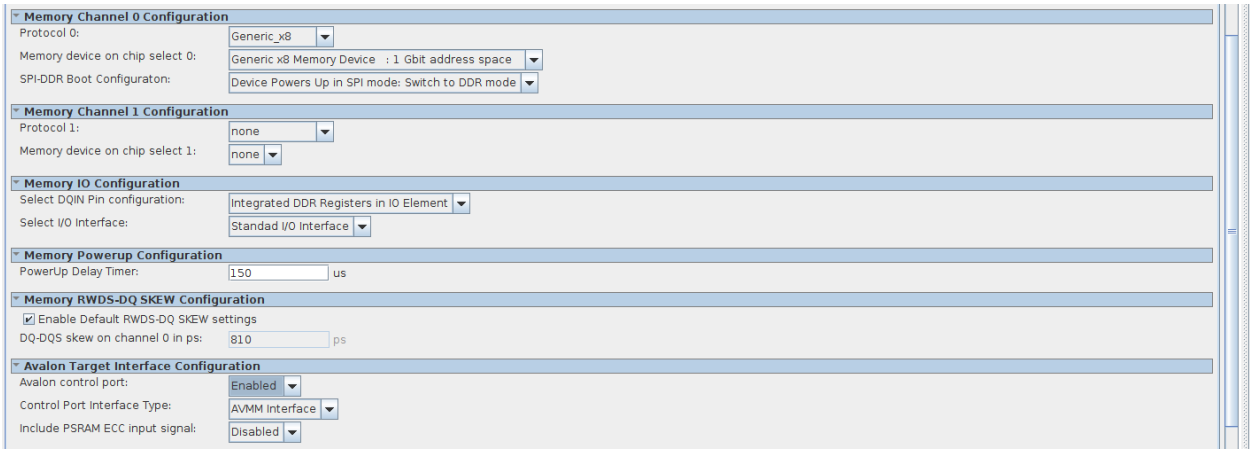
2) Generic Mode implementation

In this mode, the user configures the controller in Generic mode. This means that the user can connect any memory device supported by the controller without re-compiling the Quartus project. Both SLL xSPI-MBMC controller and the memory devices will need to be configured / initialized through software.

In generic mode, the user can test different devices without the need to re-compile the Quartus design. Software will be used to initialize SLL xSPI-MBMC controller according to the device present on the FPGA board.

Pleas note that at the current stage, SLL xSPI-MBMC controller cannot automatically detect which memory device(s) are present on board setup. When testing memory devices of the same family, the user can easily write a small route in software to automatically detect which device is present on the FPGA and program SLL xSPI-MBMC accordingly.

Below is an example of configuring SLL xSPI-MBMC IP in Generic Mode.



Memory Channel 0 Configuration is set to Generic_x8 mode.

- Set **Memory Device 0 Protocol** to **Generic_x8**.
- Set **Memory Device 0 Protocol Generic_x8** to **1Gbit Address space** (**this is user defined)
- Set **Memory Device 0 SPI Boot Config accordingly**

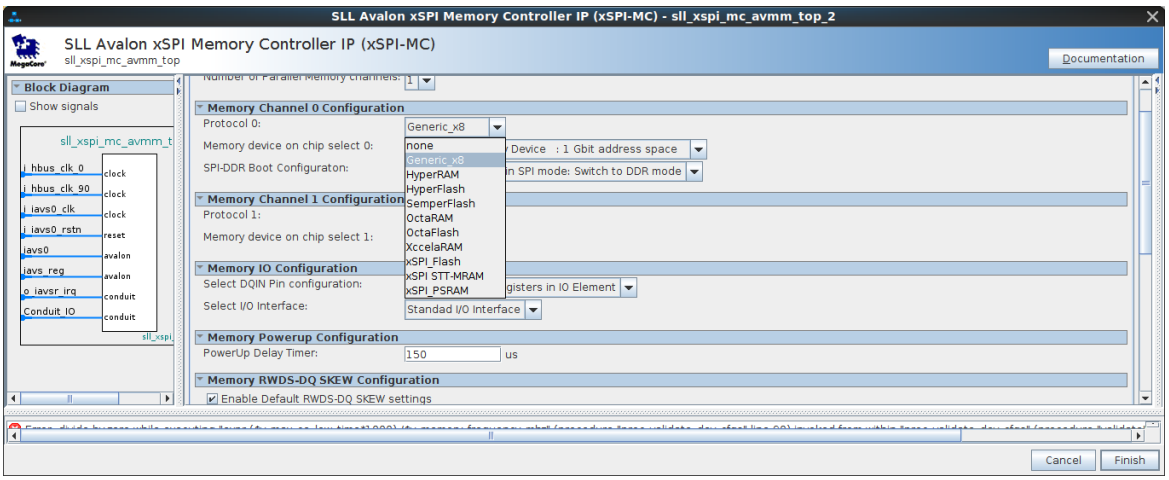
Memory Channel 1 Configuration is set to None

- Set **Memory Device 0 Protocol** to **None.**

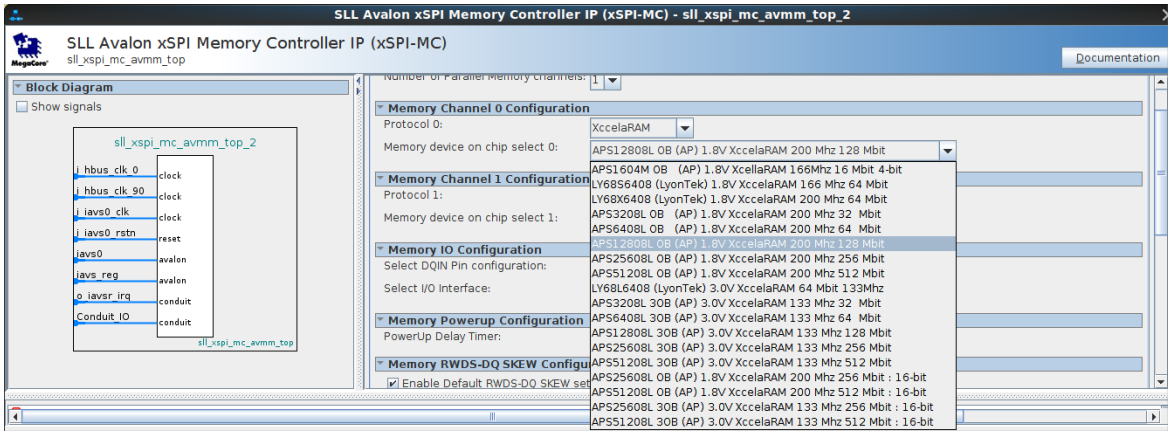
Control Port Configuration is set to enabled

2.1 Memory Channel 0/1 Configuration

- **Memory Device 0/1 Protocol :** Selects the PseudoRAM / Flash memory Protocol according to the device present on the FPGA board. Currently, SLL xSPI-MC supports the following protocols : HyperRAM, HyperFlash, SemperFlash, OctaRAM, OctaFlash, XccelaRAM, xSPI PSRAM and xSPI Flash. If no device is present select **None**.

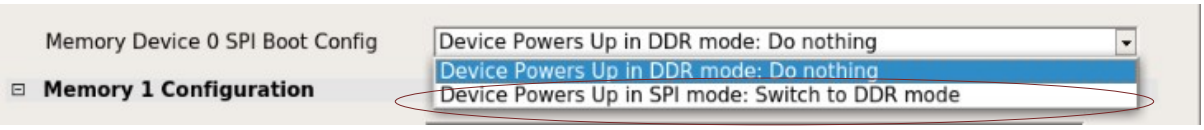


- **Memory Device 0/1 Name :** This is a list of supported devices based on the protocol selected in the **Memory Device Protocol** field. Select the device present on the FPGA board. The example below shows a list of devices supported under the XccelaRAM Protocol.



- Kindly contact Synaptic Labs' (info@synaptic-labs.com) in case the PSRAM/Flash device present on your FPGA board is not available in the list.

- **Memory Channel 0/1 SPI Boot Config** *(where applicable)* : Most memory devices supported by SLL xSPI-MC IP powers up in x8 DDR mode. However, certain memory devices powers up in SPI (x1) mode after reset. In this cases, the memory device needs to be configured to x8 DDR mode. This can be achieved by selecting the
  - **Device Powers Up in SPI mode : Switch to DDR** field. SLL xSPI-MC IP will automatically configure the memory device to x8 DDR after reset.



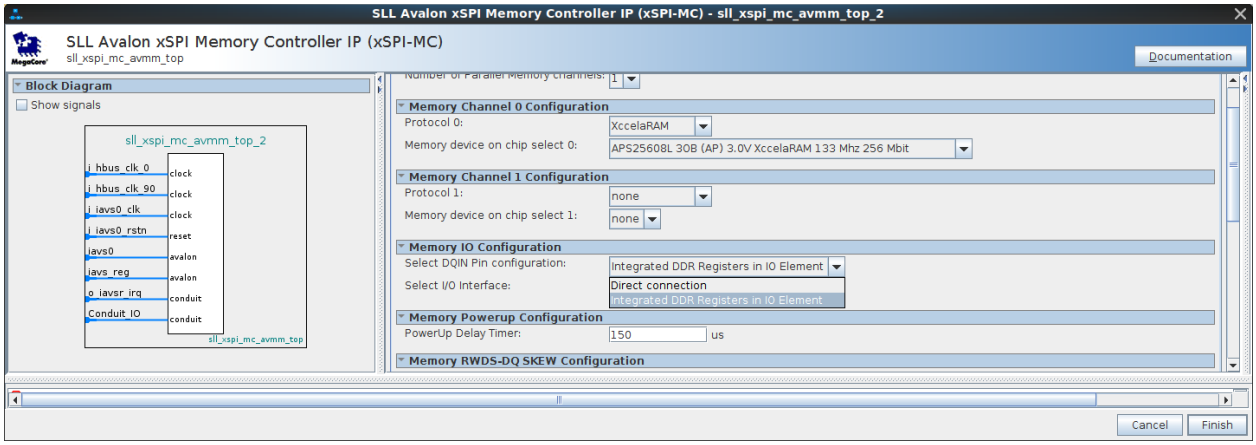
2.2.2 Memory IO Configuration

Select DQIN Pin Configuration

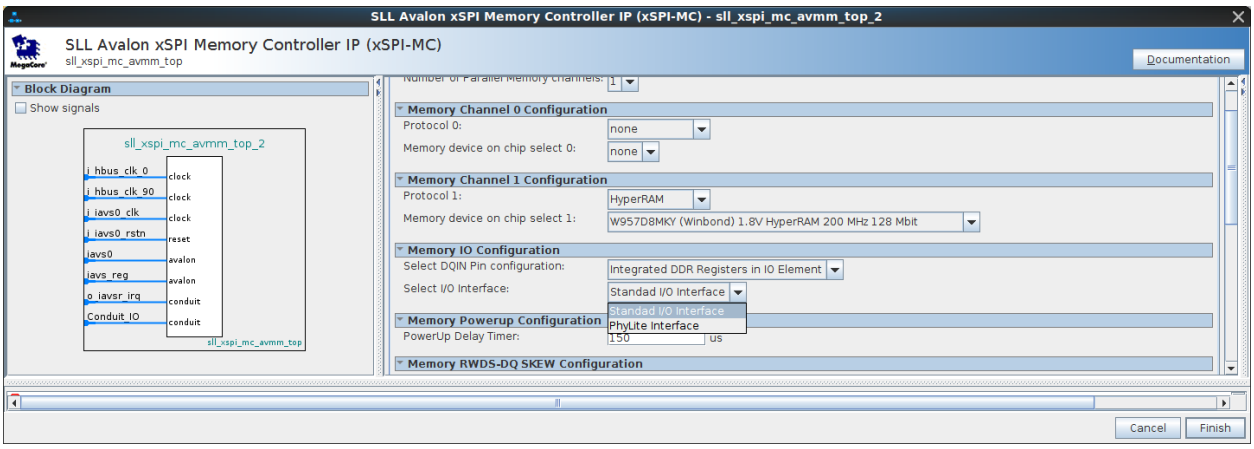
The user can can either select

- Direct Connection or
- Integrated DDR Register in the IO Element (preferred option)

The Integrated DDR feature can be used for all device families. The Integrated DDR feature instantiates the PHY in a way that enables the DDR FF inside the I/O Elements. The Integrated DDR Register in the IO Element option allows for potential higher clock speeds to be achieved.



Select I/O interface Configuration



The user can can either select

- Standard I/O interface or
- Phylite interface

The Standard I/O interface should be used with C10LP, MAX10 and Cyclone V FPGA devices. The Phylite interface should be used with C10GX, ARRIA 10 and Stratix 10 FPGA devices.

The Phylite interface uses Intel's PhyLite for Parallel Interfaces. Kndly contact Syntaptic Labs' for more info.

2.2.3 Memory Powerup Configuration

All PSRAM/Flash devices have a power-up time period between the release of the hardware reset and the first memory access.

The Powerup Delay timer variable controls the powerup delay timer between the release of the reset signal and the first memory access.

Please check the data sheet and use the correct value for the power-up time period for the memory devices present on the board.

2.2.4 RWDS-DQ Skew Configuration

When the **[x] Enable Default RWDS-SKEW** is checked, the RWDS-DQ skew will be automatically calculated.

When the **[ ] Enable Default RWDS-SKEW** option is left unchecked, the user can enter the RWDS-DQ skew manually.

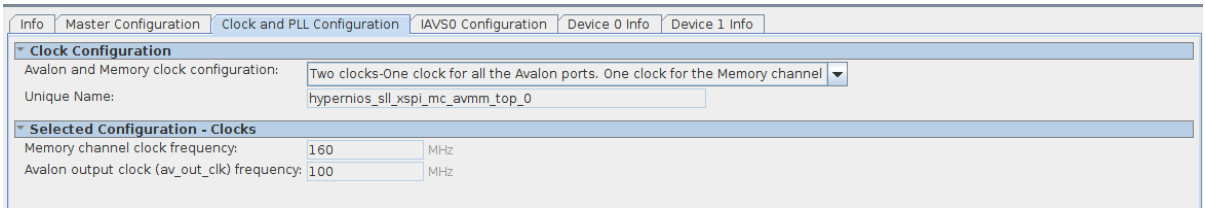
The value will be used to set the correct timing parameters in the timing constraint script automatically generated by SLL MBMC IP.

2.2.5 Avalon Target interface Configuration

When enabled, a secondary Avalon-MM port is created.
This can be used for programming Flash devices or accessing pseudoRAM registers.

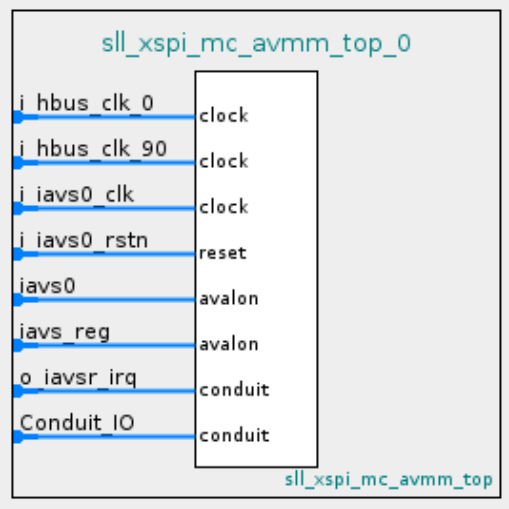
2.3 Configuring SLL xSPI-MC IP in the Clock and PLL Configuration Tab

SLL xSPI-MBMC IP is configured in external PLL mode. Internal PLL mode is no longer supported. The option is no longer visible in the configuration settings.



2.3.1 External PLL mode

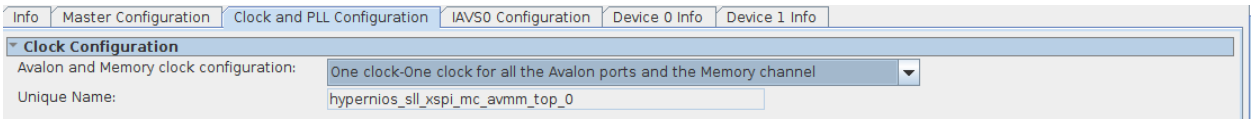
The different Avalon-MM master/slave clocks and PSRAM/Flash Clocks in the design are generated by an external PLL.



In the external PLL mode, S/Labs' includes 3 clock signals

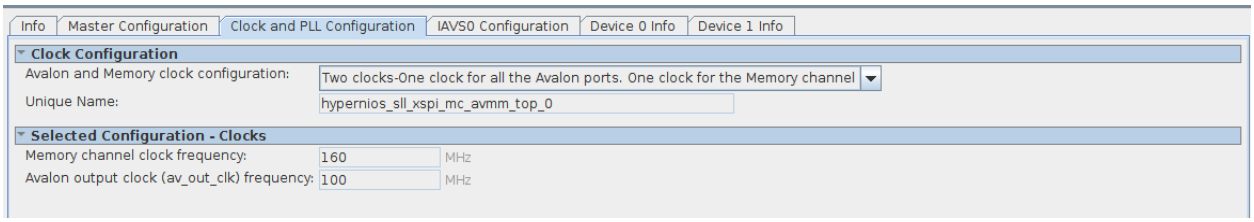
i_hbus_clk_0 : input clock driving the Hyperbus controller
i_hbus_clk_90 : input clock driving the Hyperbus controller
Operates at same frequency as i_hbus_clk_0
Phase shifted 90 degrees
i_iavs_clk : input clock driving the Avalon-MM interface

2.3.1.1 For Single Clock Operation



Select this option when the PSRAM/Flash memory channel is operating at the same frequency as the Avalon Memory slave interface.

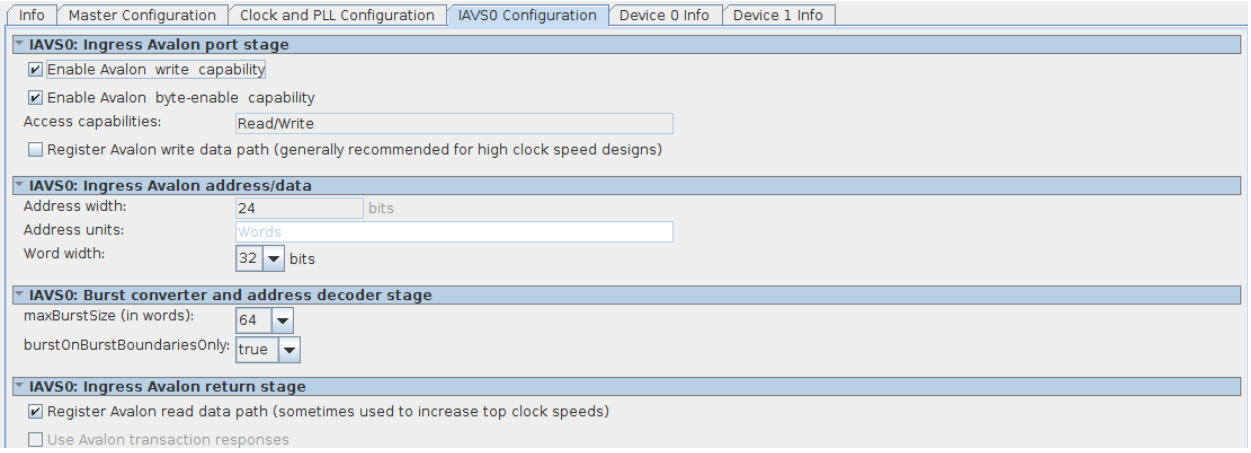
2.3.1.2 For Dual Clock Operation



Select this option when the PSRAM/Flash memory channel is operating at a different frequency than the Avalon Memory slave interface.

2.4 Configuring SLL xSPI-MC IP in the IAVS0 Configuration Tab

The Avalon port slave has its own port configuration tab. This offers great flexibility and enables the interface to be configured under different parameters.



2.4.1 Configuring Ingress Avalon port slave parameters

The "**Enable Avalon write capability**" feature is enabled by default and adds the Avalon write signal conduits on this slave interface. If the Avalon slave interface is a read only connection, such as the Nios II instruction master, the write signals can be disabled by un-ticking this box.

The " **Enable Avalon byte-enable capability**" feature is enabled by default and adds the Avalon byeenable signal conduit on this slave interface. For bus masters that do NOT employ the byte-enable signals, this feature can be turned off.

The "**Register Avalon Write data**" feature allows the registration of the write data signal on the Avalon slave to potentially increase clock speeds and ease place-and-route.

2.4.2 Configuring the burst count on the Ingress Avalon port slave

The " **maxBurstSize (in words)**" GUI label select the maximum burst length of read and/or write burst memory transfer requests on this port in words. The value of **maxBurstSize** must be set to the value of the largest **maxBurstSize** of all Avalon bus masters interfaces that are connected to this slave interface.

When a burst size greater than 1 is selected, additional parameters become available to configure the burst behaviour.

If one or more of the Altera Avalon bus master interfaces connected to this slave port employs **linewrapBursts=true**, then set the **linewrapBursts** of this slave port to true.

An additional parameter **burstOnBurstBoundariesOnly** becomes available.

- Set the value of **burstOnBurstBoundariesOnly** to true for the slave port when all bus master interfaces connected to this port have their **burstOnBurstBoundariesOnly** parameter set to true.
- Otherwise, set the value of **burstOnBurstBoundariesOnly** to false for the slave port.

Example : if the Nios II processor is configured with

- Instruction cache enabled and the "Add burstcount signal to instruction master " enabled. (This will configure the Nios II instruction master with a burst size of 8 and linewrapBursts=true).
- Data cache enabled and the "Add burstcount signal to data master " enabled. (This will configure the Nios II data master with a burst size of 8, linewrapBursts=false and burstOnBurstBoundariesOnly=true).
- and we are connecting both the Instruction Master and the Data master to SLL Multiple Bus Memory Controller Avalon-MM slave

then we set **maxBurstSize (in words) to 8**, and **linewrapBursts=true.**

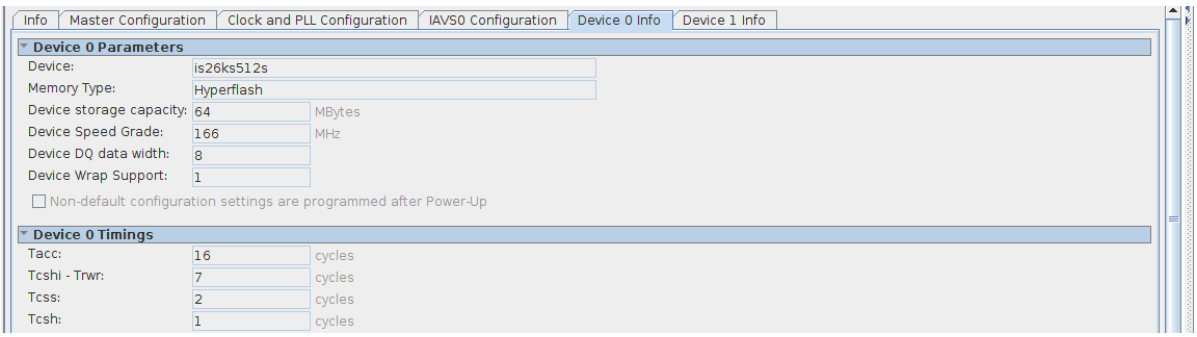
2.4.3 Configuring the Ingress Avalon port slave return path

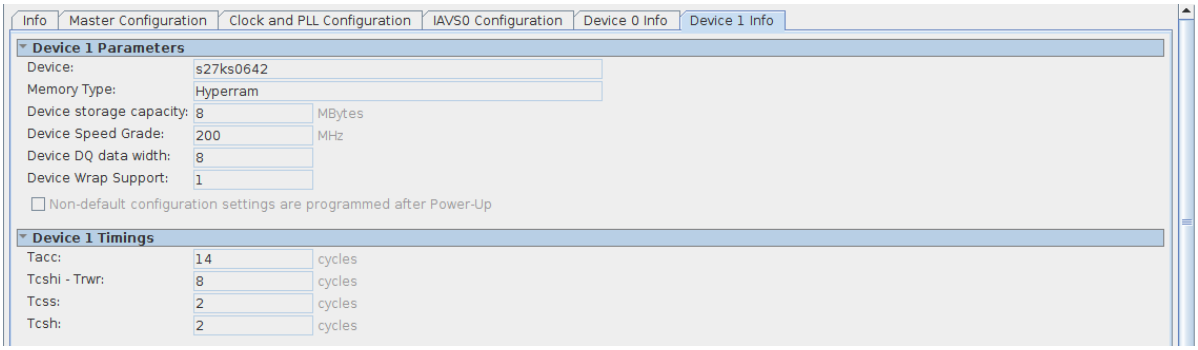
The "**Register Avalon Read data**" feature allows the registration of the read data signal on the Avalon slave to potentially increase clock speeds and eases place-and-route.

The "**Use Avalon Transaction Response**" feature adds the Avalon transaction response conduit.

2.5 Configuring SLL xSPI-MC IP in the Device 0/1 Tab

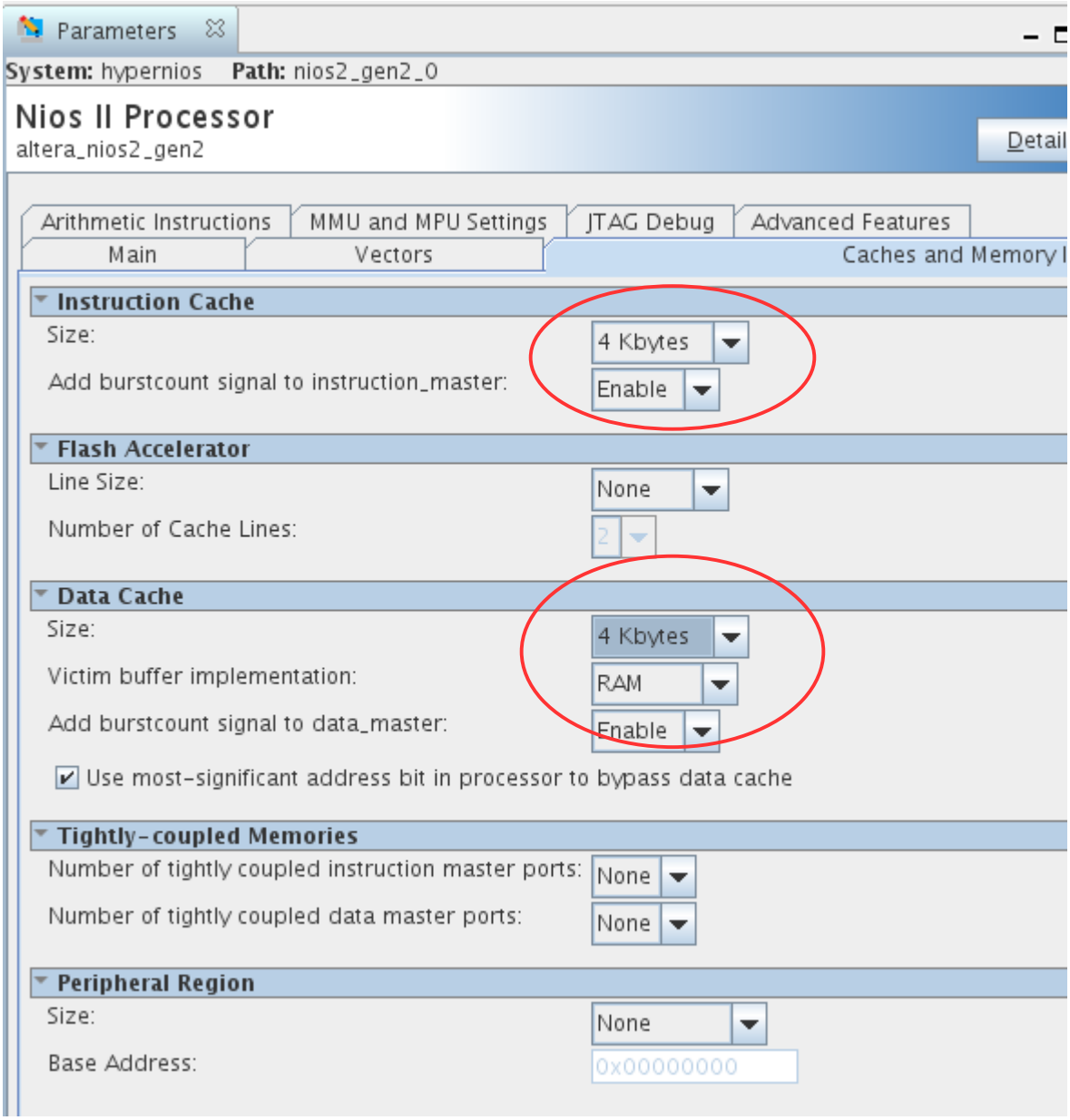
The **Device 0/1 Parameters and Timing** fields shows information about the memory device selected on Chip Select 0. The user cannot change any of the fields.





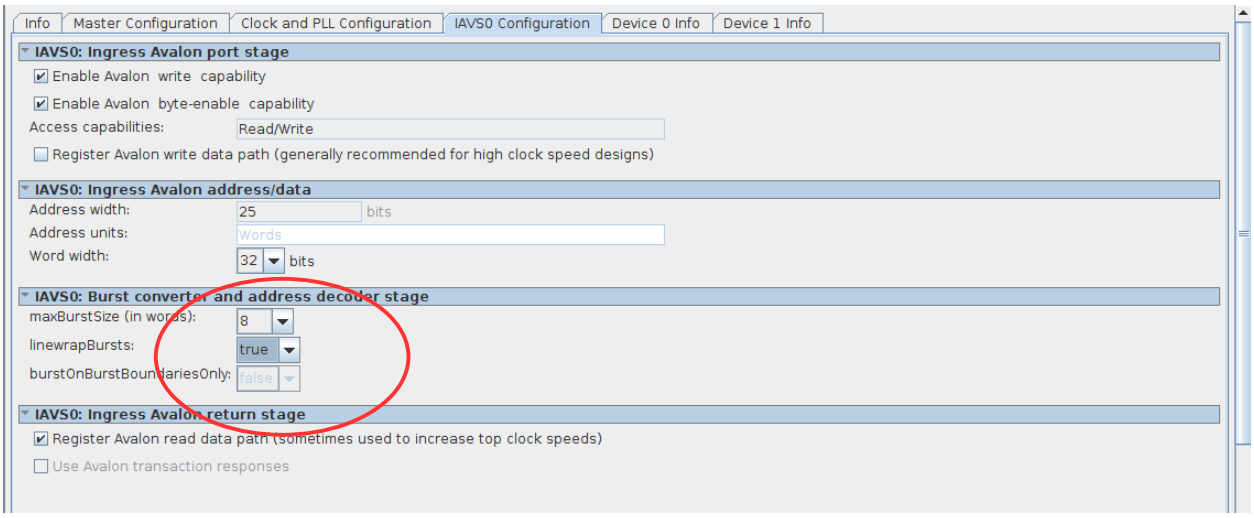
On startup or after reset, SLL xSPI Memory Controller will automatically program the selected PSRAM/Flash Memory control register with the chosen settings.

3.0 Connecting SLL xSPI Memory Controller to the NIOS II/f embedded processor



Configure the Nios II/f processor with Instruction/Data caches enabled and **burstcount** signals enabled.

Configure the SLL Multi Bus Memory Controller Avalon slave (IAVS0 configuration tab) with a **burstsize** of 8 and **linewrap burst** enabled.



3.1 SLL xSPI Memory Controller with external PLL Configuration

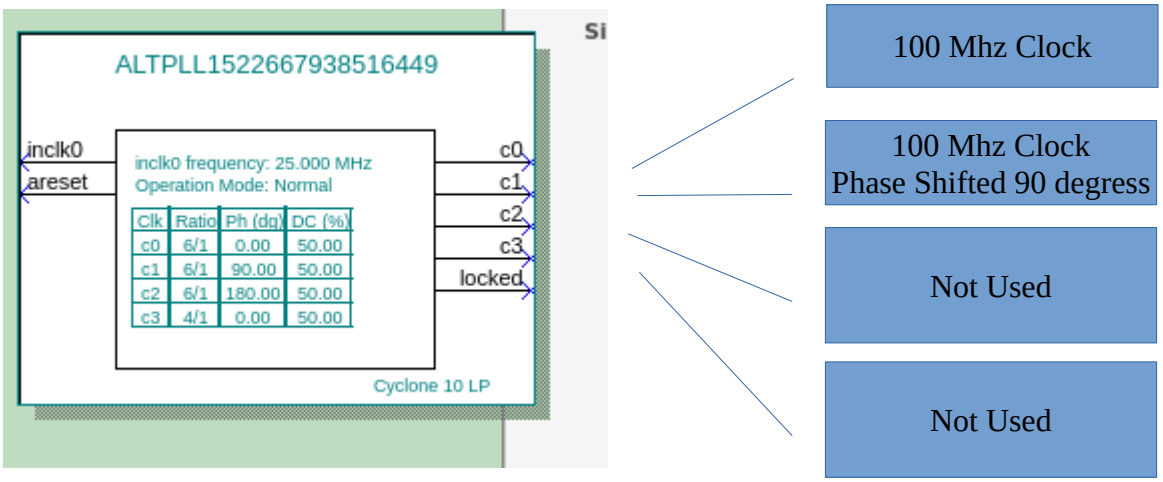
With the external PLL configuration, SLL xSPI-MC IP requires 4 clocks :

- **i_hbus_clk_0** : clock driving the xSPI-MC Memory Channel
- **i_hbus_clk_90** : clock for driving some signals on the xSPI-MC Memory Channel. It operates at the same frequency as **i_hbus_clk_0** but is phase shifted 90 degrees
- **i_iavs0_clk** : clock driving the Avalon-MM interface. When SLL xSPI-MC IP is configured to run at a single clock speed, this clock is connected to **i_hbus_clk_0** clock.

3.1.1 Option A – Same Clock for the PSRAM/Flash memory channel an Avalon-MM channel.

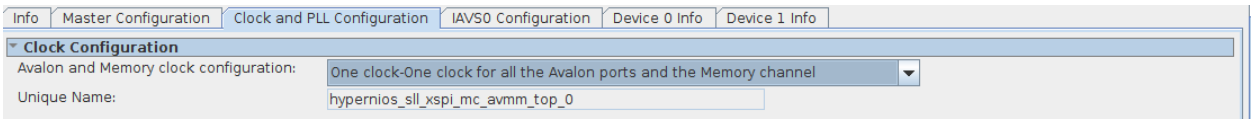
This configuration shows how to connect SLL xSPI-MC IP so that the PSRAM/Flash memory channel operates at the same frequency as the Avalon-MM bus interface. The advantage of this configuration is lower circuit area.

3.1.2 Clocking (PLL) Wizard Configuration



The figure below shows a typical example of configuring the Clocking wizard . In this case, the clocks for the PSRAM/Flash channel and Avalon-MM interface channels are all set to 100 Mhz.

3.1.3 SLL xSPI-MC Configuration



In this example, SLL xSPI-MC IP is configured with :

- **Avalon and Memory clock configuration** : **One clock**

3.1.4 SLL xSPI-MC wiring

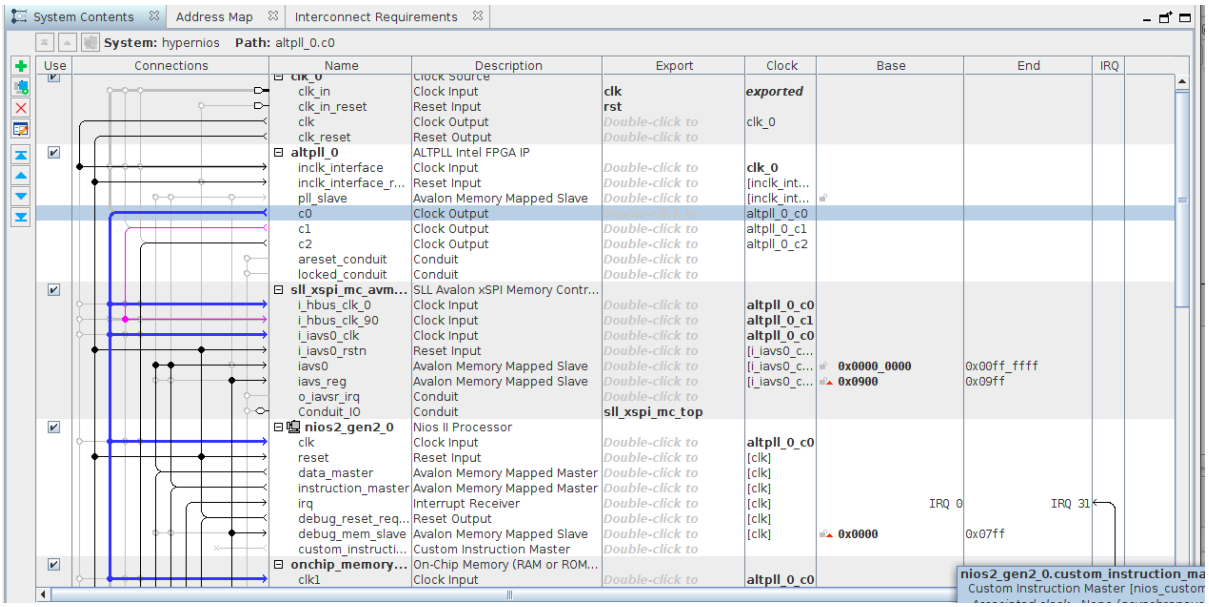
Altera PLL output clock 0 (c0)

- Connect to i_hbus_clk_0 and i_iavs0_clk on SLL xSPI-MC IP
- Connect to other Avalon-MM slaves and masters clock sinks

Altera PLL output clock 1 (c1)

- Connect to i_hbus_clk_90 on SLL xSPI-MC IP

Note how i_hbus_clk_0 and i_iavs0_clk are connected to the same clock.

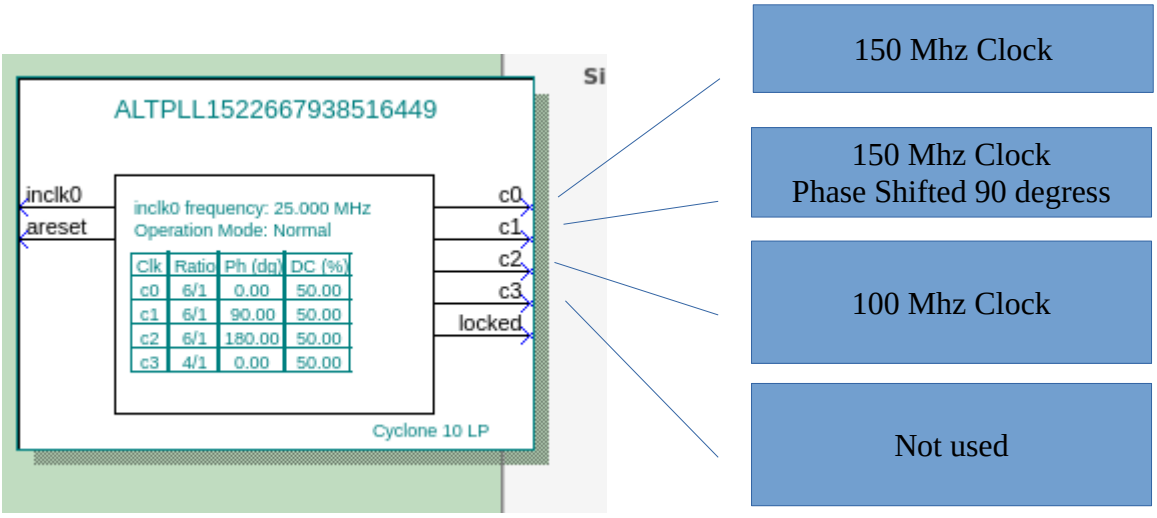


3.2 Option B – Different Clocks for the PSRAM/Flash memory channel and Avalon-MM channel .

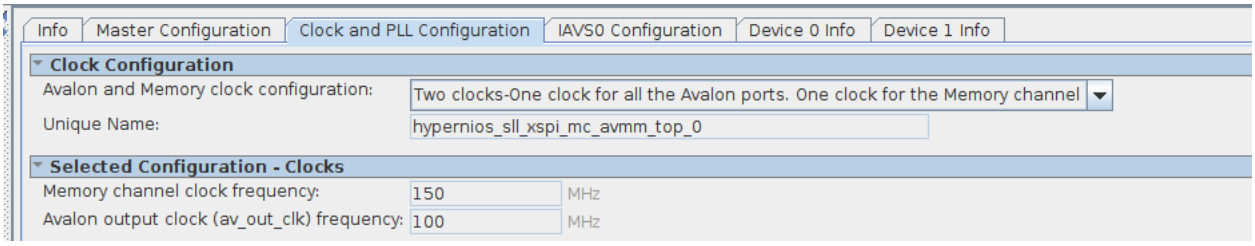
This configuration shows how to connect SLL xSPI-MC IP so the PSRAM/Flash memory channel operates at a different clock frequency then the Avalon-MM bus interface.

3.2.1 Clocking Wizard Configuration

The figure below shows a typical example of configuring the Clocking wizard . In this case, the clocks for the PSRAM/Flash channel are all set to 150 Mhz, while the clock for the Avalon-MM interface channel is set to 100 Mhz.



3.2.2 SLL MBMC Configuration



In this example, SLL xSPI-MC IP is configured with :

- **Avalon and Memory clock configuration** : **Two clocks**

3.2.3 SLL xSPI-MC wiring

Altera PLL output clock 0 (c0)

- Connect to i_hbus_clk_0 on SLL xSPI-MC IP

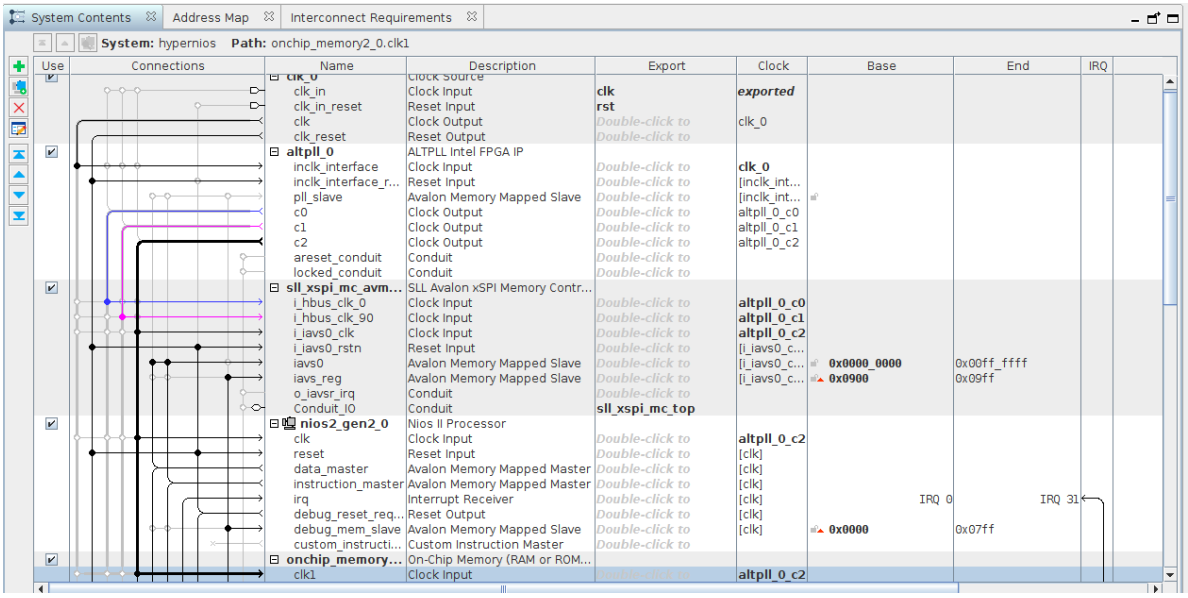
Altera PLL output clock 1 (c1)

- Connect to i_hbus_clk_90 on SLL xSPI-MC IP

Altera PLL output clock 3 (c2)

- Connect to i_iavs0_clk on SLL xSPI-MC IP
- Connect to other Avalon-MM slaves and masters clock sinks

Note how i_hbus_clk_0 and i_iavs0_clk are NOT connected to the same clock.



4.0 AVMM Limitations

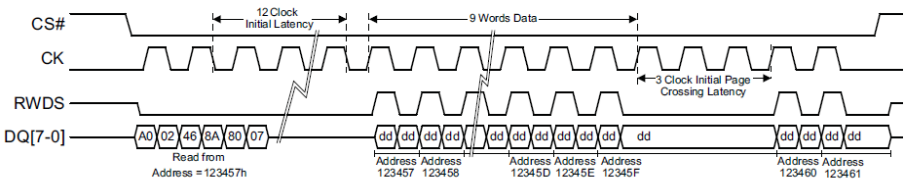
No limitations.

4.1 Device Related Limitations

Hyperflash 26KS/26KL – SemperFlash 26HS/26HL

- These memory modules have a strange way of stopping the RWDS during the Page Boundary clock crossing.

Figure 7. Read Transaction Crossing a Page Boundary[12, 13, 14, 15, 16]



- Calculation of the Page boundary depends on a number of factors :

To calculate latency when crossing a page boundary, use the following formula:

```
if ((PS - LTCY) < ADDR & (SP -1))
  {
  ((ADDR & (SP -1)) - PS + LTCY)
 }
else
  {0}
```

where:

PS = page size = 16 words

SP = sub-page size = 8 words

LTCY = latency

ADDR = target address

- This is not supported in SLL xSPI-MC IP with an AXI4 interface. To mitigate this issue, it is advised that AXI Addresses needs to be aligned to the burst transfer size.

- SLL offers an AXI4 Bridge to mitigate this issue

AXI4 Wrap Access mode limitations on Hyperram 3.0 (S27KL/KS0643, S27KL/KS1283)

- Hyperram 3.0 memory devices do not support direct wrap access mode .
- Hence AXI4 WRAP access cannot be used on SLL xSPI-MC AXI data bus
- To mitigate this issue, it is advised that AXI Addresses needs to be aligned to the burst transfer size.
- SLL offers an AXI4 Bridge to mitigate this issue
- You can contact SLL if your design require AXI4 WRAP access with Hyperram 3.0 memory devices.

AXI4 Wrap Access mode limitations on xSPI-Flash

- xSPI-Flash memory devices do not support direct wrap access mode.
- Hence AXI4 WRAP access cannot be used on SLL xSPI-MC AXI data bus
- To mitigate this issue, it is advised that AXI Addresses needs to be aligned to the burst transfer size.
- SLL offers an AXI4 Bridge to mitigate this issue
- You can contact SLL if your design require AXI4 WRAP access with xSPI- Flash memory device.

PSRAM Chip Select Low limitation

- some PSRAM devices have a maximum Chip Select Low limitation. This comes into play when large bursts are issued to the memory.
- Large bursts can be split using SLLAXI4 Bridge to mitigate this issue

Everspin STT-MRAM device

- minimum write transfer is 16-bit (2 bytes). You can contact SLL if your design require byte write access.
- maximum write burst is limited to the page size if Configuration Register 8 Bit 0 (NOR Flash like mode) is set to 1 . If for some reason, the user needs to use this feature, SLL offers an AXI4 Bridge so one can issue write bursts larger than the page size

Nonvolatile Configuration Register 8					
Address	**0x0008**				
Bit	**Op**	**Name**	**Settings**	**Description**	**Notes**
7	RW	Erase Bit Value	1 = Erase with '1' (default) 0 = Erase with '0'	Determines data state for Erase operations	
6:2	RW	TBD	TBD		
1	RW	RPE	1 = Reset Pin Enabled (default) 0 = Reset Pin Disabled	When the Reset pin is disabled, the RESET # will be ignored. The device may be reset with Soft Reset or JESD Reset. In the DFN package option, the RESET# signal is disabled automatically when the device is configured in Quad SPI mode (Register 0).	
0	RW	Write mode	1 = Persistent Memory Mode enabled (default) 0 = Persistent memory mode disabled	Persistent Memory mode or NOR Flash like mode is selected with this bit.	3

5.0 Signal Description

5.1 Memory Bus Signal Interface

Hyperbus Signal	Description	Comments	Board consideration
MB_CLK0p	Pseudo Differential clock pair 0	To be connected to pin CK on Memory device 0.	Connect to memory CK signal
MB_CLK0n	Pseudo Differential clock pair 0 (used with 1.8V devices)	To be connected to pin CK# on Memory device 0.	Connect to memory CK# signal when required.
MB_CLK1p	Pseudo Differential clock pair 1	Additional Clock pair	Not connected
MB_CLK1n	Pseudo Differential clock pair 1	Additional Clock pair	Not connected
MB_CS0n	Chip select device 0	To be connected to pin CS# on Memory device 0.	Connect to Flash CS# signal. Leave unconnected if Flash not present
MB_CS1n	Chip select device 1	To be connected to pin CS# on Memory device 1.	Connect to PSRAM CS# signal
MB_Wpn	Disables writes to Flash memory devices (Write protect)	To be connected to pin WP# on any Flash device.	Connect to Flash WP# if required. *Currently not used.*
MB_RWDS	Read strobe/Write mask signal	To be connected to pin RWDS on all memory devices.	Connect to PSRAM / FLASH RWDS signal
MB_DQ	Data bus (8-bit)	To be connected to DDR Data pins on all memory devices.	Connect to PSRAM / Flash Dq signals
MB_INTn	Interrupt from Flash Devices to FPGA.	To be connected to pin INT# on all Flash device.	Connect to Flash INT# signal. *Currently not used*
MB_RST0	Reset from Flash Devices to FPGA	To be connected to pin RSTO on all Flash devices.	Connect to Flash RSTO signal when required
MB_RSTn	Reset to Memory Devices	To be connected to pin RESET# on all memory devices.	Connect to memory RST# signal when required

Optional Pullup Resistor *(not used for the PSRAM Only Configuraton)*

MB_INTn and MB_RSTO signals signals of the Hyperbus memories are open drain output without a pull-up resistor. Therefore, when these signals are used in the design, it is suggested that a pull-up resistor is inserted either by setting the appropriate constraint on the input FPGA pad or adding a resistor on the board itself.

5.1 Memory Clock Signal Interface

Pll Signal	Description	Comments
i_hbus_clk_0	Memory Channel Clock	Memory Clock
i_hbus_clk_90	Memory Channel Clock (90 phase shift)	90 degrees phase shift

5.2 AVMM Clock/reset interface

AXI signal	Description	Comments
i_iavs0_aclk	AVMM clock source	
i_iavs0_aresetn	AVMM reset source	

5.3 AVMM Interface

Avalon-MM signal	Description
i_iavs0_do_wr	Avalon Write Request
i_iavs0_do_rd	Avalon Read Request
i_iavs0_addr	Write-Read address.
i_iavs0_burstcount	Burst length. The burst length gives the exact number of transfers in a burst.
i_iavs0_byteenable	Write strobes. This signal indicates which byte lanes hold valid data. There is one write strobe bit for each eight bits of the write data bus.
i_iavs0_wdata	Write data
i_iavs0_wait_request	Avalon wait request. This signal indicates whether the slave can accept a write or read request
i_iavs0_rvalid	Read valid. This signal indicates that the return read data is valid
i_iavs0_rdata	Return read data

5.4 AVMM register Control Interface

Avalon-MM signal	Description
i_iavsr_do_wr	Avalon Write Request
i_iavsr_do_rd	Avalon Read Request
i_iavsr_addr	Write-Read address.
i_iavsr_wdata	Write data
i_iavsr_rdata	Return read data

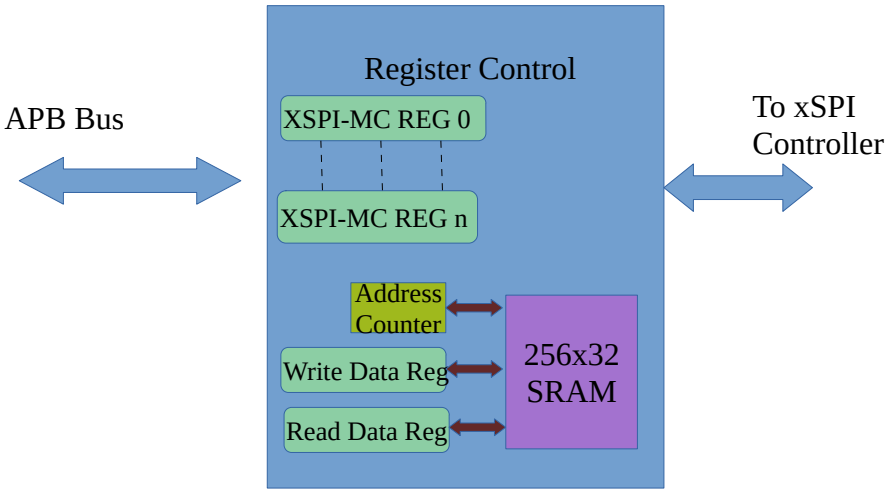
There are no wait states for the read and write transfers.

6.0 APB Register Control Port

SLL xSPI-MC includes an APB3-MM slave interface (s00_apb). This can be used for creating memory regions, configuring the xSPI controllers according to the devices present, programming Flash memory devices and also accessing PseudoRAM registers. Please note that read and write transactions over the AXI memory port always have precedence over memory transfer requests issued by this control port.

The Register Control Port contains :

- a small bank of Read/Write Register for controlling logic and reading status of the core
- a small memory buffer (256x32 SRAM) for Flash programming



The Register Control Port includes a small memory buffer (256x32 SRAM). This is particularly used for programming the xSPI flash devices as well as reading multi-word registers from memory devices.

Multiple access to the xSPI-MC_WDATA Register will fill data into consecutive locations in the SRAM.
Multiple access to the xSPI-MC_RDATA Register will get data from consecutive locations in the SRAM.

The Address counter for accessing the SRAM memory is

- incremented every time the xSPI-MC_WDATA Register is accessed
- incremented every time the xSPI-MC_RDATA Register is accessed.
- cleared automatically when xSPI-MC_CMD is written
- cleared automatically when xSPI-MC_CAD_LEN is written
- cleared automatically when the memory controller acknowledge the operation
- cleared automatically at the end of a read memory operation

When the APB Register Control Port is enabled, an APB-MM slave interface becomes present (s00_apb). This can be used for programming Flash and accessing PseudoRAM registers. Please note that read and write transactions on the AXI memory port always have precedence over memory transfer requests issued by this control port.

Please note that Synaptic Labs' offers device drivers for all supported memory devices. Kindly contact Synaptic Labs' for the latest drivers.

6.1 Register MAP

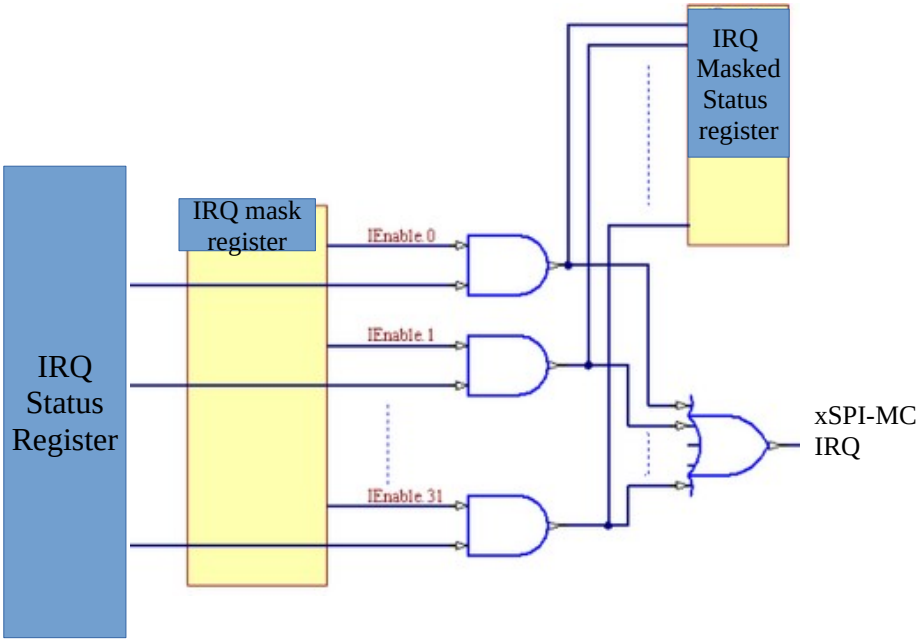
6.1.1 xSPI-MC Command Address Control registers

Address	Register Name	Direction	Comment
0x10	XSPI-MC-CAD LEN	Write Only	Number of 32-bit words to be transferred
0x14	xSPI-MC_CAD_HI	Write Only	Upper 16-bit data CA[47:32] (command/address phase **)
0x18	xSPI-MC_CAD_LOW	Write Only	Lower 32-bit CA[31:0] (command/address phase **)
0x1C	xSPI-MC_WDATA	Write Only	32-bit write data
0x2C	xSPI-MC_RDATA	Read Only	32-bit return read data

Address	Register Name	Direction	Comment
0x40	xSPI-MC_CMD	Write only	Command register
0x5C	xSPI-MC_STATUS	Read only	Status register

6.1.2 xSPI-MC Info and interrupt registers

Address	Register Name	Direction	Comment
0x44	xSPI-MC_THREAD	Read/Write	Thread ID Register
0x48	xSPI-MC_IRQ_MASK	Read/Write	This is used to determine whether or not an active interrupt source should generate an interrupt request. Each data bit that is HIGH sets the corresponding bit in the mask register.
0x4C	xSPI-MC_IRQ_CLR	Write Only	This is used to determine which active interrupt source should be cleared. Each data bit that is HIGH clears the corresponding bit in the IRQ status register. All other bits of the IRQ Status register are unaffected.
0x4C	xSPI-MC_IRQ_STATUS	Read Only	The interrupt status indicates whether or not the appropriate interrupt source is active.
0x50	XSPI-MC DEC_ERR	Read Only	Errors related to AXI Decode
0x54	XSPI-MC FIFO ERR	Read Only	Errors related to FIFOs
0x64	XSPI-MC IRQ MASKED status	Read Only	The interrupt status indicates whether or not the appropriate interrupt source is active. The interrupt is generated when the respective IRQ Mask Bit and IRQ Status Register bit are both active


IRQ Masked Status register
IRQ mask register
IEnable 0
IEnable 1
IEnable 31
IRQ Status Register
xSPI-MC IRQ

6.1.3 xSPI-MC Memory Device 0/1 Config registers

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

Address	Register Name	Direction	Comment
0x80	xSPI-MC_DEV_0_CFG	Read/Write	CS0# Memory Configuration Register
0x84	xSPI-MC_DEV_0_TIM	Read/Write	CS0# Memory Timing Register
0x88-0x8C	xSPI-MC_DEV_0_CMD	Read/Write	CS0# Memory Command Register
0x90	xSPI-MC_DEV_1_CFG	Read/Write	CS1# Memory Configuration Register
0x94	xSPI-MC_DEV_1_TIM	Read/Write	CS1# Memory Timing Register
0x98-0x9C	xSPI-MC_DEV_1_CMD	Read/Write	CS1# Memory Command Register

6.1.4 xSPI-MC Memory Device 0/1 Memory Partition registers

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

Address	Register Name	Direction	Comment
0xA0	xSPI-MC_DEV_0_MBA	Read/Write	CS0# Memory Base Address Register
0xB0	xSPI-MC_DEV_1_MBA	Read/Write	CS1# Memory Base Address Register

6.1.5 xSPI-MC MODE Control registers

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

Address	Register Name	Direction	Comment
0xC0	xSPI-MC_MODE_CTRL	Write Only	Mode Control Register
0xC4	xSPI-MC_DLL_CTRL	Read/Write	DLL delay control

6.1.6 xSPI-MC SPI-TO-DDR Control registers

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

Address	Register Name	Direction	Comment
0xC8	xSPI-MC_SPI_TO_DDR0	Write Only	Switch from x1 to x8 DDR command
0xCC	xSPI-MC_SPI_TO_DDR1	Write Only	Switch from x1 to x8 DDR command

Some memory devices power up in SPI (x1) mode. SLL xSPI-MC contains a simple SPI controller that can send commands to the memory device in SPI (x1) mode. The command is defined by setting the XSPI_MC_SPI_TO_DDR0/1 registers

This feature can be used during initialization to switch the memory device from SPI (x1) mode to X8 DDR with Read Strobe so it can be used with SLL xSPI-MC controller. ** This feature is not used if the design has a QSPI controller. In this case, at power up, the QSPI controller sends SPI (x1) commands to the memory device before switching to SLL xSPI-MC controller.

6.1.7 xSPI-MC JEDEC251 Rst register

Address	Register Name	Direction	Comment
0xD0	xSPI-MC_JEDEC251_Rst	Write Only	SLL xSPI-MC Jedec Reset Control Register

6.1.8 xSPI-MC ID registers

Address	Register Name	Direction	Comment
0xF0	xSPI-MC_ID_0	Read Only	SLL xSPI-MC ID 0
0xF4	xSPI-MC_ID_0	Read Only	SLL xSPI-MC ID 1

6.2 Register Definitions

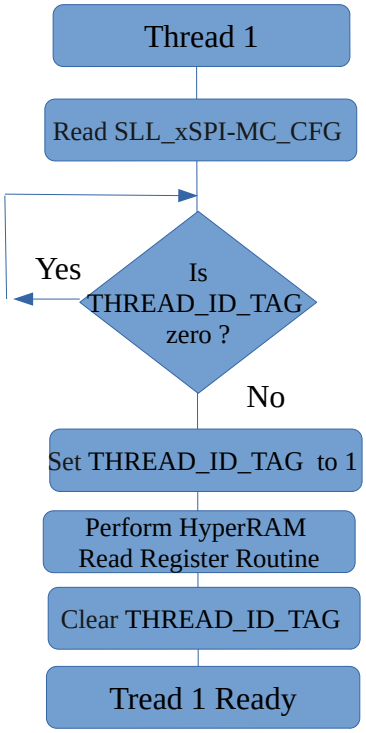
6.2.1 SLL_xSPI-MC_THREAD Register *(optional)*

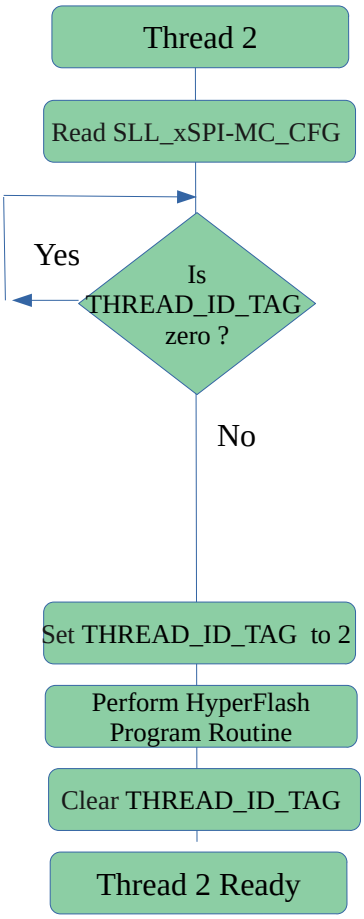
bit	Name	Description
7:0	THREAD_ID_TAG	Can be used to signal which thread is currently in use. Value of 0 means that the Register Control Port is currently not In use.
11:8	Reserved	

Note : It is very important that operations though the Register Control Port are accessed by only one thread at a time. The THREAD_ID_TAG field can be used to control different threads that need to perform operations through the Register Control Port.

As an example, let's suppose that there are 2 independent threads running

- Thread 1 wants to read the HyperRAM registers
- Thread 2 wants to program the HyperFlash





6.2.2 SLL_xSPI-MC_IRQ Registers

SLL_xSPI-MC_IRQ Mask Register

This is used to determine whether or not an active interrupt source should generate an interrupt request. Each data bit that is HIGH sets the corresponding bit in the mask register.

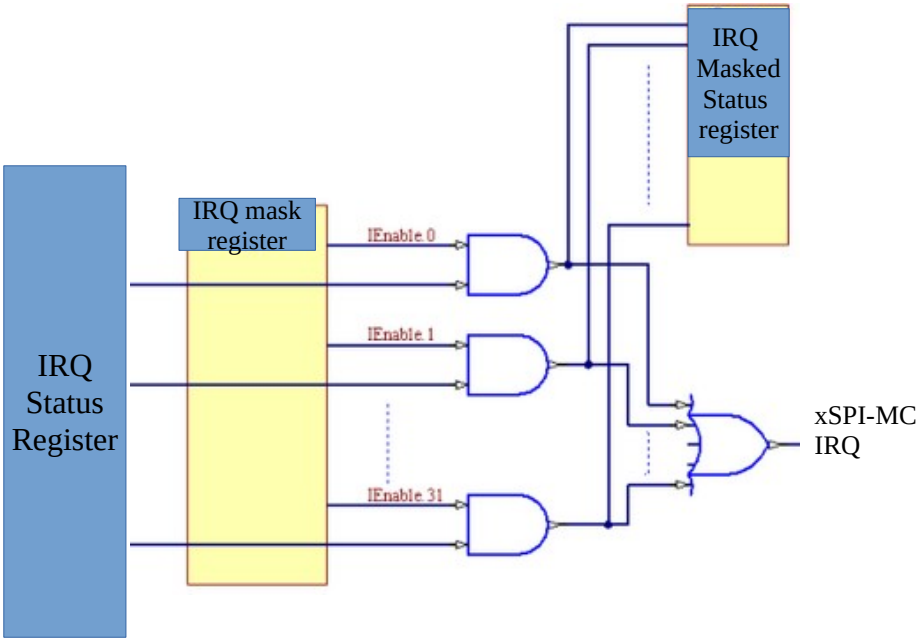
bit	Name	Description
0	RdataTimeout	When set, a Timeout Error on Read Data Path will set the corresponding bit in the xSPI-MC_IRQ Masked register
1	DecErr	When set, an Error at the AXI Decode stage will set the corresponding bit in the xSPI-MC_IRQ Masked register
2	ExtIRQ	When set, a **negative** edge on the External IRQ Pin status will set the corresponding bit in the xSPI-MC_IRQ Masked register
3	FifoErr	When set, an Error on any of the Fifos detected will set the corresponding bit in the xSPI-MC_IRQ Masked register
4	ExtEcc	When set, a **positive** edge on the External ECC signal will set the corresponding bit in the xSPI-MC_IRQ Masked register
5	Cad_Ecc_1bit_err	When set, an ECC 1-bit Error on the Register RAM during software Register Read Command will set the corresponding bit in the xSPI-MC_IRQ Masked register
6	Reg_Ecc_1bit_err	When set, an ECC 1-bit Error on the Register RAM during APB RDATA read will set the corresponding bit in the xSPI-MC_IRQ Masked register
7	AXI memory read	When set, a read on the AXI Data channel will set the corresponding bit in the xSPI-MC_IRQ Masked register
8	AXI memory write	When set, a write on the AXI Data channel will set the corresponding bit in the xSPI-MC_IRQ Masked register
9	Cad_Ecc_2bit_err	When set, an ECC 2-bit Error on the Register RAM during software Register Read Command will set the corresponding bit in the xSPI-MC_IRQ Masked register
10	Reg_Ecc_2bit_err	When set, an ECC 2-bit Error on the Register RAM during APB RDATA read will set the corresponding bit in the xSPI-MC_IRQ Masked register

SLL_xSPI-MC_IRQ Status, xSPI-MC_IRQ Masked Status

This register shows the raw and masked interrupt sources generated by the controller. An interrupt will be thrown is if the respective interrupt mask bit is set.

- the xSPI-MC IRQ status register is the raw interrupt status
- the xSPI-MC IRQ **masked** status register shows the interrupt status when the corresponding bits of both the IRQ status registers and the IRQ mask registers are active.

bit	Name	Description
0	RdataTimeout	Timeout Error on Read Data Path
1	DecErr	Error at AXI Decode stage detected. Register xSPI-MC_Decode Status will supply more information about what's causing this fault.
2	ExtIRQ	Negative Edge detected on the External IRQ Pin signal
3	FifoErr	Error on any of the Fifos detected. Register xSPI-MC_Fifo Status will supply more information about what's causing this fault.
4	ExtEcc	Positive Edge detected on the External ECC Pin signal
5	Cad_Ecc_1bit_err	ECC 1-bit Error on the Register RAM during software Register Read Command detected
6	Reg_Ecc_1bit_err	ECC 1-bit Error on the Register RAM during APB RDATA read detected
7	AXI memory read	AXI Data channel memory read transfer detected
8	AXI memory write	AXI Data channel memory write transfer detected
9	Cad_Ecc_2bit_err	ECC 2-bit Error on the Register RAM during software Register Read Command detected
10	Reg_Ecc_2bit_err	ECC 2-bit Error on the Register RAM during APB RDATA read detected



6.2.3 SLL xSPI-MC_Fifo Status Register

This register gives information about errors generated by the controller's fifos. If any of these bits become high, Interrupt Status Register bit 3 (FifoErr) will be set.

bit	Name	Description
0	RxInfoPushErr	Read Data Path Info Buffer Push Error
1	RxInfoPopErr	Read Data Path Info Buffer Pop Error
2	RxFifoPushErr	Read Data Path Fifo Push Error
3	RxFifoPopErr	Read Data Path Fifo Pop Error
4	RxFifoEcc1BitErr	Read Data Path RAM Ecc 1-bit
5	RxFifoEcc2BitErr	Read Data Path RAM Ecc 2-bit
7:6	Rsv	Reserved
8	TxInfoPushErr	Write Data Path Command Buffer Push Error
9	TxInfoPopErr	Write Data Path Command Buffer Pop Error
10	TxFifoPushErr	Write Data Path Fifo Push Error
11	TxFifoPopErr	Write Data Path Fifo Pop Error
12	TxFifoEcc1BitErr	Write Data Path RAM Ecc 1-bit
13	TxFifoEcc2BitErr	Write Data Path RAM Ecc 2-bit
15:14	Rsv	Reserved

6.2.4 SLL xSPI-MC_Decode Status Register

This register gives information about errors generated during memory address decoding. If any error occurs, the respective bit is toggled and Interrupt Status Register bit 1 (DecErr) will be set.

bit	**Name**	**Description**
0	AxiWr	AXI command is write
1	AxiRd	AXI command is read
2	RstActive	Controller in reset state error
3	CtrlNotActive	Controller in not in active state error
4	SetupNotReady	Controller in not ready to accept any commands error
5	NoRegion	AXI address point to no region error
6	MultipleRegion	AXI address point to multiple regions error

6.2.5 SLL_xSPI-MC_STATUS Register

bit	Name	Description
8:0	IrqStat	Reflects the value of the IRQ status register
15:9	rsv	Reserved
16	PwrUp	When 1, denotes that memory controller is still powering up
17	ReadValid	Following a single Read command issued through this register port, this bit indicates that the operation is completed and data in the xSPI-MC_RDATA register is valid. This bit will be cleared when SLL_xSPI-MC_STATUS register is read over the APB interface
18	CmdActive	Indicates that the Register Command is still active or pending. This bit will be cleared when SLL_xSPI-MC_STATUS register is read over the APB interface
19	ReadAllValid	Following a multi-word Read command issued through this register port, this bit indicates that the operation is completed and data in the xSPI-MC_RDATA RAM is valid. This bit will be cleared when SLL_xSPI-MC_STATUS register is read over the APB interface
20	SpiCmdActive	When 1, denotes that memory controller is still performing a SPI to DDR command. This status bit is only important when sending SPI commands over the **SPI-TO-DDR** registers.
21	Flash RSTOn	Reflects the value of the external Flash RSTOn signal. (Optional)
22	Core Rst Mode	Reflects the value of the Core Reset Mode (Bit 8) in the xSPI-MC_CTRL register
23	Dev Rst Mode	Reflects the value of the Device Reset Mode (Bit 9) in the xSPI-MC_CTRL register
24	IRQ	Reflects the value of the IRQ signal generated from the IRQ Status / IRQ mask logic

It is not necessary to poll the SLL_xSPI-MC_STATUS Register bit 16 (PwrUP) to start accessing the HyperFlash / PseudoRAM devices. The xSPI-MC AXI interface forces signals AXI awready and AXI arready low until power-up is completed. The AXI read/write request will the taken once power-up is ready.

6.2.6 SLL_xSPI-MC_CMD Register

bit	Name	Desciption	Supported Values
0	RW Mode	Read or Write operation	0 : Write 1 : Read
1	RegSpace	Selecting whether access is to the memory address space or register address space. The latter is typically used with PSRAM to program registers	0 : Memory Address Space 1 : Register Address Space
2	CS	Memory Device Selection	0 : Select Device 0 1 : Select Device 1
5:3	CmdAddrOnly	Total number of CK pulses before terminating. When not 0, only CMD-ADDR phase is issued on the bus. Should only be used with • Write mode (RW Mode=0) • CA_CK_ctrl=0	0 : normal operation 1 : 1 CK with No Data 2 : 2 CK with No Data 3 : 3 CK with No Data 4-7 : not used. Behaviour undefined
6	SIZE	Data Transfer Size. Please note that for read operation, xSPI controller always uses 32-bit access.	0 : 16- bit write – only allowed for single access 1 : 32-bit write / 32-bit read
7	SPI	SPI Mode - not used	Should be set to 0
15:8	LAT	Read or Write latency	If 0, the latency timing value in the **xSPI-MC_DEV_x_TIM** (T_RD_Tacc or T_WR_Tacc) will be used. Other wise this value will be used.
17:16	CA_CK_cycles	Controls number of clock cycles for Command/Address phase (to be used in conjunction with RW Data access)	0 : 1 CK 1 : 2 CK 2 : 3 CK 3 : not used. Behaviour undefined
19:18	CA_CK_ctrl	Controls CA CK cycles	0 : CA is always 3 CK cycles 1 : CA depends on CA_CK_cycles 2 : not used. Behaviour undefined 3 : CA depends on CA_CK_cycles and should only be used with Write mode (RW Mode=0)

When this register is written, a command is issued to the memory controller.

Chip Select Control

- When memory is accessed through the AXI Read or Write channel, Chip Select is controlled by **xSPI-MC_DEV_x_MBA** and **xSPI-MC_DEV_x_CFG.size** registers
- When memory is accessed through the Software Control channel, Chip Select is controlled by **SLL_xSPI-MC_CMD.cs** (bit 2) register

Command mapping in the CMD-ADDR field

- When memory is accessed through the AXI Read, command mapping in the CMD-ADDR phase is controlled by **xSPI-MC_DEV_x_CMD** register and wrap mode access in the AXI ARBURST field.
  - If ARBURST = WRAP and ARLEN = 7, then **xSPI-MC_DEV_x_CMD .**CMD_WRAP_RD is used
  - for all other combinations, **xSPI-MC_DEV_x_CMD.**CMD_LINEAR_RD is used
- When memory is accessed through the AXI Write, command mapping in the CMD-ADDR phase is controlled by **xSPI-MC_DEV_x_CMD** register and wrap mode access in the AXI WRBURST field.
  - If AWBURST = WRAP and AWLEN = 7, then **xSPI-MC_DEV_x_CMD .**CMD_WRAP_WR is used
  - for all other combinations, **xSPI-MC_DEV_x_CMD.**CMD_LINEAR_WR is used
- When memory is accessed through the Software Control channel, command mapping in the CMD-ADDR phase is controlled by **SLL_xSPI-MC_CAD_HI** register

Number of clocks in CMD-ADDR Phase

- When memory is accessed through the AXI Read or Write channel, CMD-ADDR phase is always 3 cycles
- When memory is accessed through the Software Control channel, CMD-ADDR phase is determined by **xSPI-MC_CMD Register. See table below**

RW Mode	CmdAddrOnly	CA_CK_ctrl	CA_CK_cycles	CA cycle on the bus	comment
Read/write	0	0	0	3	With rw data
Write	1, 2, 3	0	0	1, 2, 3	No data
Write	1, 2, 3	1, 2, 3	0, 1, 2, 3	undefined	Do not use
Read	1, 2, 3	0, 1, 2, 3	0, 1, 2, 3	undefined	Do not use
Read	0	1	1, 2, 3	1, 2, 3	With rd data
Write	0	3	1, 2, 3	1, 2, 3	With wr data
Read	0	2, 3	0, 1, 2, 3	undefined	Do not use
Write	0	1, 2	0, 1, 2, 3	undefined	Do not use

Read Latency Access Time

- When memory is accessed through the AXI Read channel, Read access timing is taken from **xSPI-MC_DEV_x_TIM.**T_RD_Tacc register
- When memory is accessed through the Software Control channel, Read access timing is
  - **xSPI-MC_CMD.Lat** if **xSPI-MC_CMD.Lat != 0** else
  - **xSPI-MC_DEV_x_TIM**.**T_RD_Tacc**

Write Latency Access Time

- When memory is accessed through the AXI Write channel, Read access timing is taken from **xSPI-MC_DEV_x_TIM.**T_WR_Tacc register
- When memory is accessed through the Software Control channel, Write access timing is
  - **xSPI-MC_CMD.Lat** register if **xSPI-MC_CMD.Lat != 0** else
  - **1** when **xSPI-MC_CMD.Reg_space = 1** else
  - **xSPI-MC_DEV_x_TIM**.**T_WR_Tacc**

Byte Swapping

- **Read-mode : xSPI-MC_DEV_x_CFG.**Byteswap controls whether the incoming data from the memory device is manipulated before being passed to the AXI Read Channel or Software Controlled Return Data channel. If byte swapping is set, then bytes are swapped for each 16-bits of received data

- **Write-mode : xSPI-MC_DEV_x_CFG.**Byteswap controls whether data from the AXI Write data or Software Controlled Write Data channel is manipulated. If byte swapping is set, then bytes are swapped for each 16-bits of data before being sent to the memory device.

6.2.7 SLL_xSPI-MC_CAD_LEN Register

This denotes the number of 32-bit words to be transferred during a memory operation issued by the register control module. Maximum supported burst length is 128 words (512 bytes).

6.2.8 SLL_xSPI-MC_CAD_HI and SLL_xSPI-MC_CAD_LOW Registers

When accessing the register address space, these 2 register form the 48-bit Command Address field

- SLL_xSPI-MC_CA_HIGH should be set to CA[47:32]
- SLL_xSPI-MC_CAD_Low should be set to CA[31:0]

When accessing the memory address Space,

- SLL_xSPI-MC_CAD_HIGH should be set to 0
- SLL_xSPI-MC_CAD_Low should be set to the half-word aligned address
  - (*byte-aligned address >> 1).*

6.2.9 SLL_xSPI-MC_WDATA Register

Write Data sent during write operation. Multiple writes will fill consecutive locations of the Register Control SRAM memory.

6.2.10 SLL_xSPI-MC_RDATA Register

Return Read Data during a Read operation. Multiple reads will read consecutive locations in the Register Control SRAM memory.

6.2.11 xSPI-MC_DEV_x_CFG Register

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

bit	Name	Description
0	Type	0 : Flash, 1 : PSRAM
1	Byte_Swap	0 : No Byte Swap 1 : Swap Bytes in Halfwords
3:2	NC	Not Used. Must be set to 0
11:4	Size	Size (log 2) in Megabits 0x00 : Empty 0x01 : 2 Mbit 0x02 : 4 Mbit 0x03 : 8 Mbit 0x04 : 16 Mbit 0x05 : 32 Mbit 0x06 : 64 Mbit 0x07 : 128 Mbit 0x08 : 256 Mbit ...
13:12	NC	Not used. Must be set to 0
15:14	Bus_Protocol	Bus Protocol 00 : Hyperbus 01 : OctaBus 10:XccelaBus 11: xSPI Bus
19:16	NC	Not Used. Must be set to 0
21:20	Bus Size	Bus size 01 : 4-bit 10 : 8-bit 11 : 16-bit
23:22	NC	Not used. Must be set to 0
26:24	Address type	Address Type : 000 : hyperbus (x8) 001 : octabus (x8) 010 : hyperbus (x16) 011 : octabus (x16) 100 : xccela-bus (x4) 101 : xccela-bus (x8) 110 : xccela-bus (x16)

29:28	Extra Read Clock	Extra Clock cycles inserted on Read

Address Mapping on AXI memory access or software control in memory access mode (xSPI-MC_CMD.reg_space =0)

Address mapping in CMD-ADDR phase is controlled by **xSPI-MC_DEV_x_CFG** registers.

Field that control the mappings are : Bus Protocol, Bus Size, Address Protocol

The bus protocol need to match the Address protocol. **Only the combinations outlined below are supported.**

Bus Protocol	Address Protocol	Bus Size	Address on the Bus

Hyperbus	Hyperbus-x8	8	{xspi_cmd[15:12], axi_address_i[31:20]} // CA0
			{axi_address[19:4]} // CA1
			{1'b0, 2'b0, 10'b0, axi_address[3:1]} // CA2

Hyperbus	Hyperbus-x16	16	{xspi_cmd[15:12], 1'b0, axi_address_i[31:21]} // CA0
			{axi_address[20:5]} // CA1
			{1'b0, 2'b0, 10'b0, axi_address[4:2]} // CA2

OctaBus	Octabus-x8	8	{xspi_cmd[15:12], 4'b0, 2'b0, axi_address_i[31:26]} // CA0
			{axi_address[25:10], // CA1
			{axi_address[9:4], 4'b0, 2'b0, axi_address[3:1], 1'b0} // CA2

OctaBus	Octabus-x16	16	{xspi_cmd[15:12], 4'b0, 3'b0, axi_address_i[31:27]} // CA0
			{axi_address[26:11]} // CA1
			axi_address[10:5], 4'b0, 2'b0, axi_address[4:2], 1'b0} // CA2

XccelaBus	Xcella-bus-x4	4	{xspi_cmd[15:0]} // CA0
			{3'b0, axi_address[22:9]} // CA1
			{axi_address[8:1], 8'b0, 1'b0} // CA2

	Xcella-bus-x8	8	{xspi_cmd[15:0]} // CA0
			{axi_address[31:16]} // CA1
			{axi_address[15:1], 1'b0} // CA2

	Xcella-bus-x16	16	{xspi_cmd[15:0]} // CA0
			{1'b0, axi_address[31:17]} // CA1
			{axi_address[16:11], 1'b0, axi_address[10:2],1'b0} // CA2

xSPI-Bus	Xcella-bus-x8	8	{xspi_cmd[15:0]} // CA0
			{axi_address[31:16]} // CA1
			{axi_address[15:1], 1'b0} // CA2

	Xcella-bus-x16	16	{xspi_cmd[15:0]} // CA0
			{1'b0, axi_address[31:17]} // CA1
			{axi_address[16:11], 1'b0, axi_address[10:2],1'b0} // CA2

Address Mapping on RAW software memory access (xSPI-MC_CMD.reg_space =1)

Bus Protocol	Address Protocol	Bus Size	Address on the Bus

dont-care	dont-care	dont-care	{SLL_xSPI-MC_CAD_HI} // CA0
			{SLL_xSPI-MC_CAD_LO[31:16]} // CA1
			{SLL_xSPI-MC_CAD_LO[15:0]} // CA2

6.2.12 xSPI-MC_DEV_x_CMD Register

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

This is a 64-bit registers (split into 2) that defines the read/write command to be used by the controller during native read and write access through the AXI bus.

The commands can be obtained from the data sheet of the memory device being accessed.

When the controller is configured in **static mode**, this register is automatically configured at compile time (through HDL parameres). The user will not have the ability to change this register.

bit	Name	Description
15:0	CMD_LINEAR_RD	Octal DDR Fast Read command
31:16	CMD_WRAP_RD	64-byte Wrap Octal DDR Read command (** not all memory devices support this feature)
47:32	CMD_LINEAR_WR	Octal DDR Fast Write command
63:48	CMD_WRAP_WR	64-byte Wrap Octal DDR Write command (** not all memory devices support this feature)

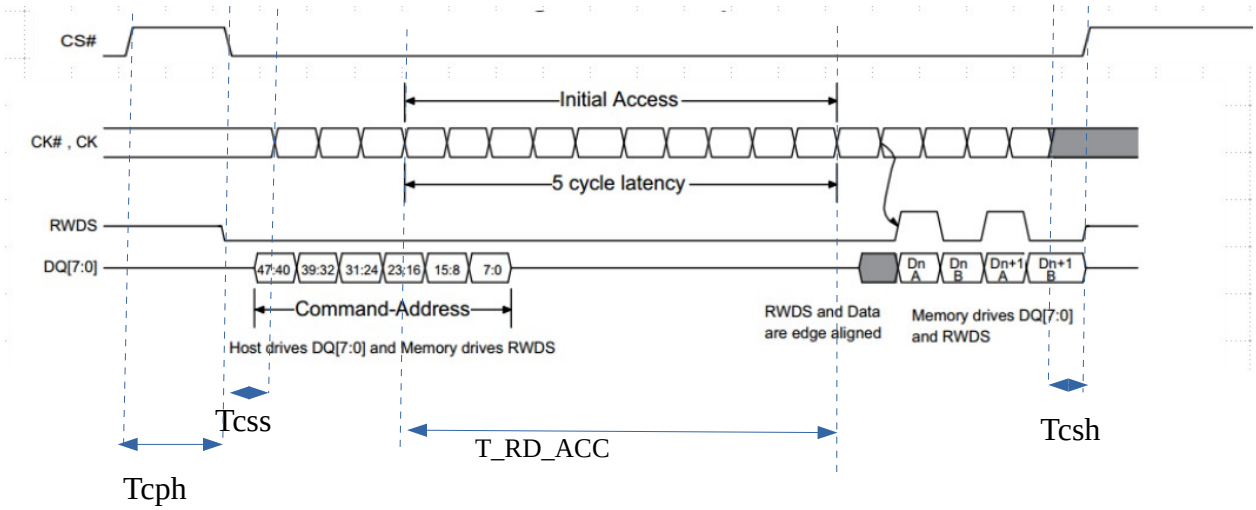
6.2.13 xSPI-MC_DEV_x_TIM Register

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

bit	Name	Description
3:0	T_CSS	Read/Write Chip Select Setup to next CK rising edge. This bit indicates CS# setup time from CS# assertion. 0x1 corresponds to 1 clock cycle, 0xF corresponds to 15 clock cycle. ** Do not set T_CSS value to 0
7:4	T_CSH	Read/Write Chip Select Hold after CK falling edge. This bit indicates CS# hold time to from CS# de-assertion. 0x0 corresponds to 1 clock cycle, 0xF corresponds to 16 clock cycle.
15:8	T_RWR	Chip Select High Between Operations or Read/Write Recovery. This bit controls the CS# high time between successive read and write operations. Select the higher value between the read/write recovery time (Trwr) or Minimum Chip Select Hgh time (Tcph)
23:16	T_RD_ACC	Initial Latency Read Access Cycle. This bit indicates initial latency code for memory read access after the second cycle in the CMD-ADDR phase.
31:24	T_WR_ACC	Initial Latency Write Access Cycle. This bit indicates initial latency code for memory write access.

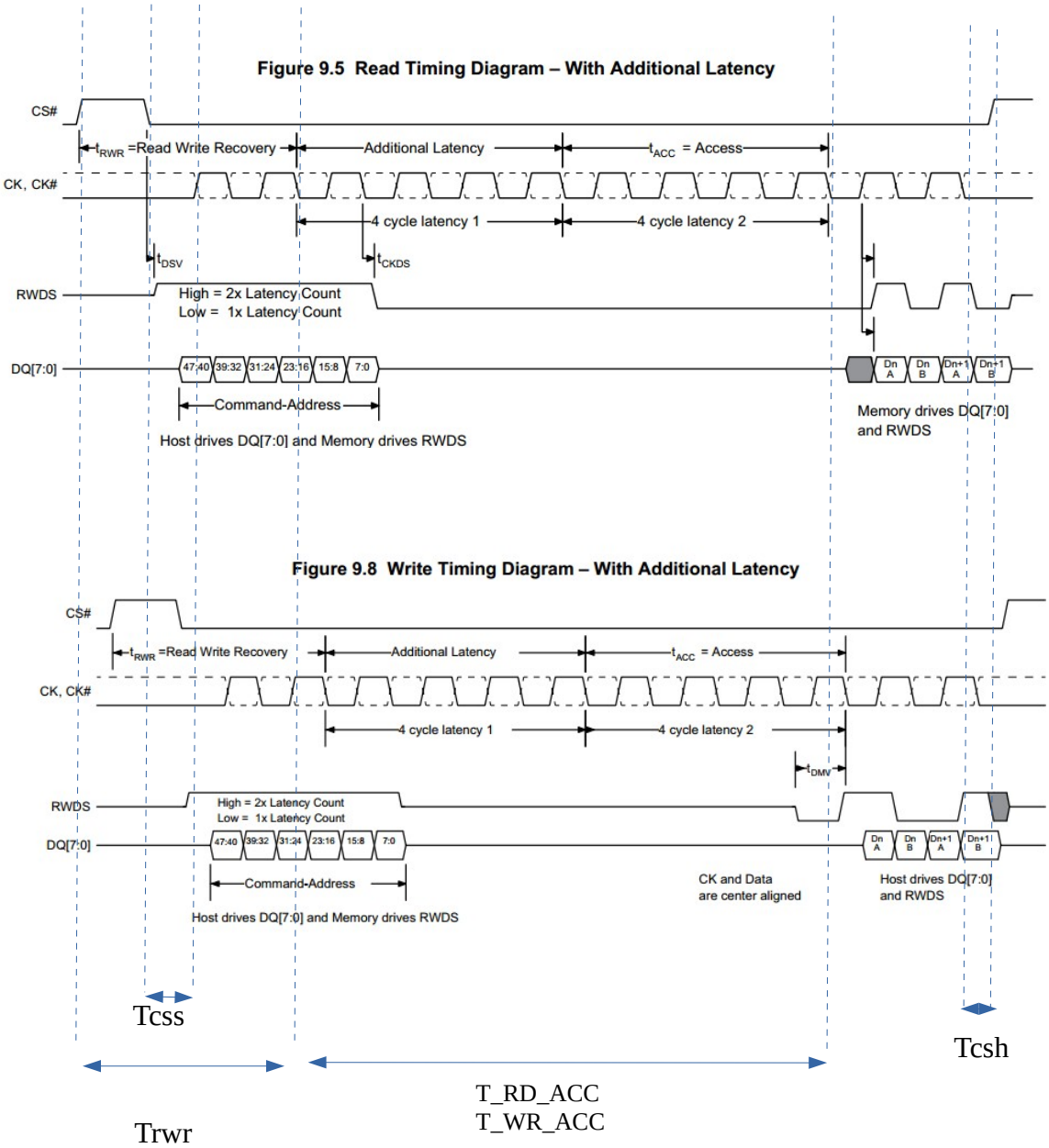
HyperFlash protocol Timing

- set T_RD_ACC in xSPI-MC_DEV_x_TIM register to the Initial Access Latency
- set T_WR_ACC in xSPI-MC_DEV_x_TIM register to 1



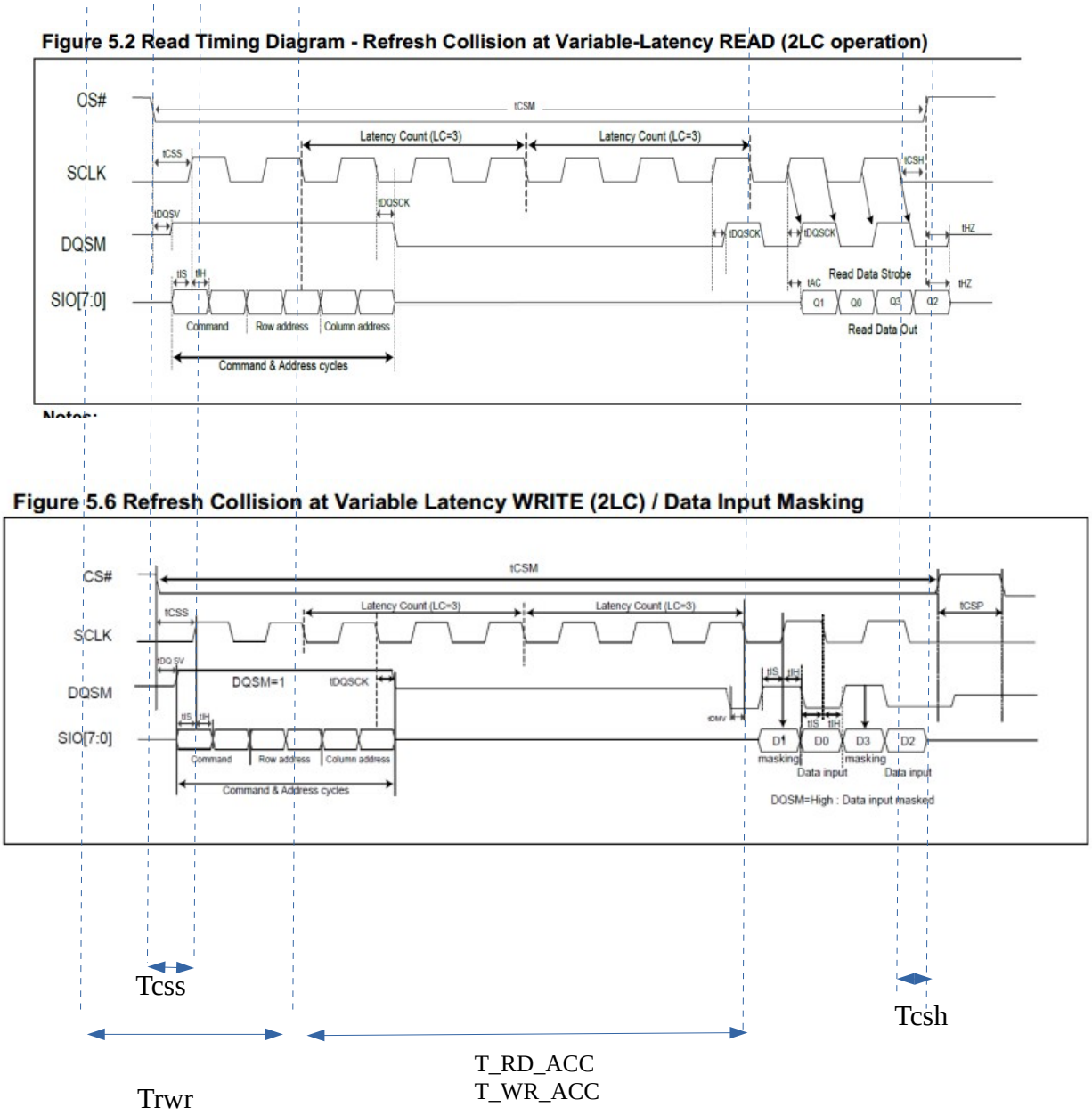
HyperRAM protocol Timing

- set T_RD_ACC in xSPI-MC_DEV_x_TIM register to 2 times Tacc
- set T_WR_ACC in xSPI-MC_DEV_x_TIM register to 2 times Tacc



OctaRAM protocol Timing

- set T_RD_ACC in xSPI-MC_DEV_x_TIM register to 2 times LC
- set T_WR_ACC in xSPI-MC_DEV_x_TIM register to 2 times LC (** depends on memory vendor. Check data sheet)



XccelaRAM protocol Timing

- set T_RD_ACC in xSPI-MC_DEV_x_TIM register to 2 times LC
- set T_WR_ACC in xSPI-MC_DEV_x_TIM register to 1 times LC

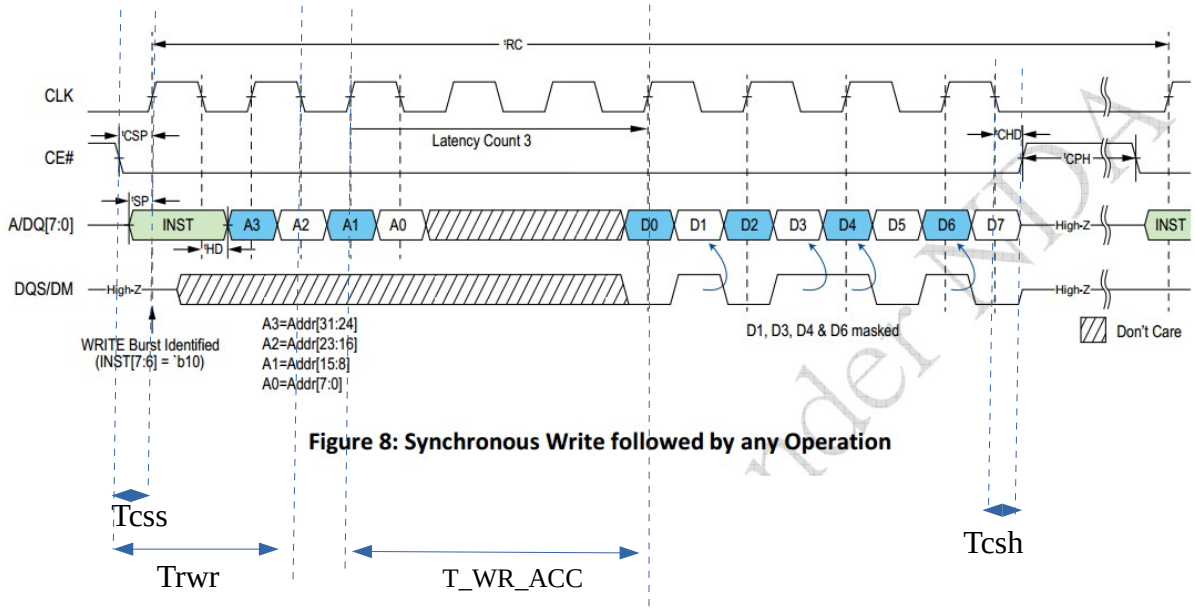


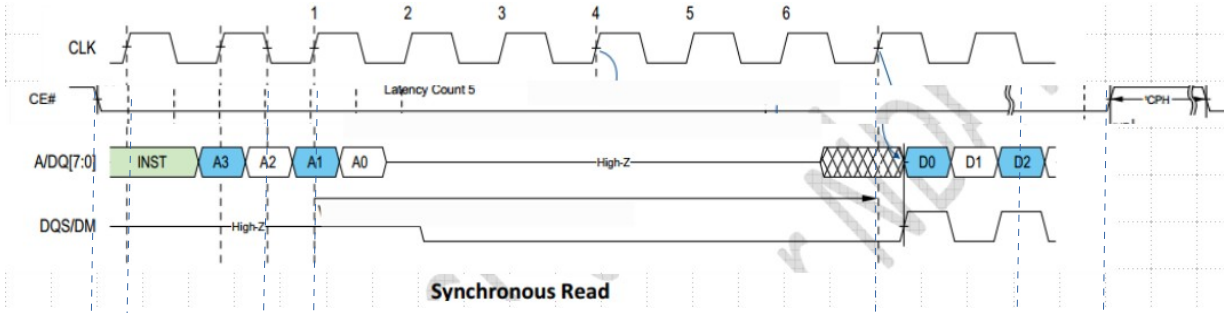
Figure 8: Synchronous Write followed by any Operation

Tcss

Trwr

T_WR_ACC

Tcsh



Synchronous Read

Tcss

Trwr

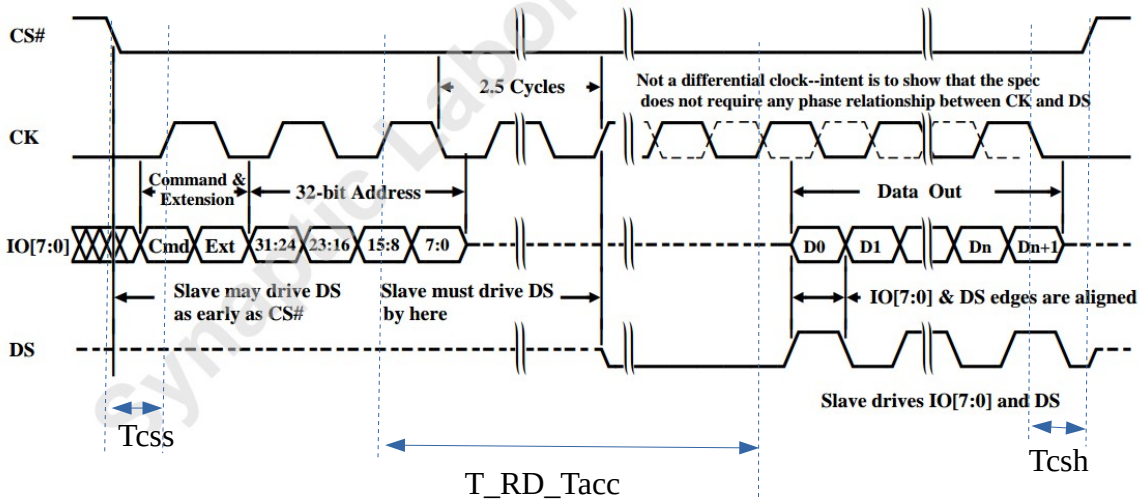
2*LC

T_RD_ACC

Tcsh

xSPI Profile 1.0 protocol Timing

- set T_RD_ACC in xSPI-MC_DEV_x_TIM register to Read access time



xSPI Profile 2.0 protocol Timing

- set T_RD_ACC in xSPI-MC_DEV_x_TIM register to 2 times LC

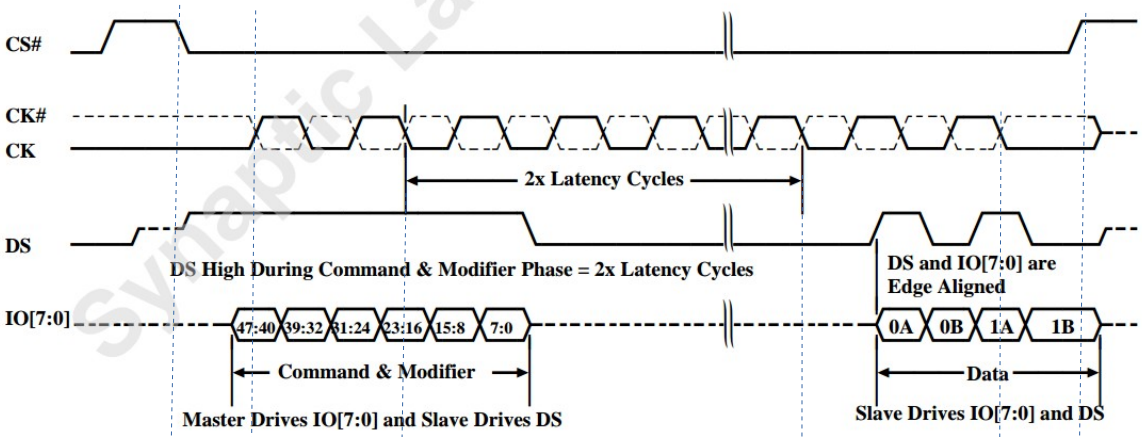
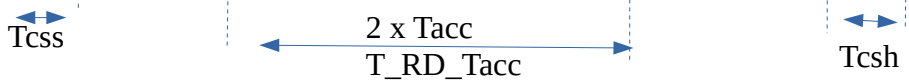


Figure 16 — Profile 2.0 8D-8D-8D Format 2.A: Command, 5-byte Address Modifier, '2x' Latency Cycles, and Read Data



6.2.14 xSPI-MC_DEV_0_MBA Register

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

This is a 64-bit register containing the start address in the AXI4 Memory Address Space for Device 0. The start address needs to be aligned to the size of the memory device 0.

The memory size of this region is specified in the xSPI-MC_DEV_0_CFG Register.

6.2.15 xSPI-MC_DEV_1_MBA Register

**** Only used when controller is programmed in generic mode**
**** For static mode, these registers are automatically generated by the GUI configurator at compile time (HDL parameres). The user will not have the ability to change this register.**

This is a 64-bit register containing the start address in the AXI4 Memory Address Space for Device 1. The start address needs to be aligned to the size of the memory device 1.

The memory size of this region is specified in the xSPI-MC_DEV_0_CFG Register.

6.2.16 xSPI-MC_CTRL Register

Bit	value	Name	Reset Value	Description
4:0	0	Core_en	0	No Core Enabled
	1			qSPI Host Controller Enabled **not used
	2			xSPI Host Controller Enabled
	3			xSPI Target Controller Enabled **not used
	4,5,6,7	Reserved		
	8	CORE_RST	1	Core Reset – for future use – not implemented
	9	DEV_RST	1	Device Reset Control
	10	SETUP_RDY	0	Setup ready bit for memory access
	11	reserved		reserved
	12	Dev 0 PD Active	0	Dev 0 Power Down Active – for future use
	13	Dev 0 PD Wake	0	Dev 0 Power Down Wakeup – for future use
	14	Dev 1 PD Active	0	Dev 1 Power Down Active – for future use
	15	Dev 1 PD Wake	0	Dev 1 Power Down Wakeup – for future use
	16	reserved		reserved
8	other	Set/Clr		Set/Clear control

To set a bit, you will need to send command **(0x100 | bit_to_set)**
To clear a bit, you will need to send command **(0x000 | bit_to_clear)**

Here are some examples :

Original value xSPI-MC_CTRL	**Write pattern**	**New value in xSPI-MC_CTRL**	**Description**
0x0000	0x102	**0x0004** (bit 2 is set)	xSPI Host Controller Enabled
0x0004	0x100	**0x0001** (bit 0 is set)	No Core_en Enabled
0x0001	0x102	**0x0004** (bit 2 is set)	xSPI Host Controller Enabled
0x0004	0x109	**0x0204** (bit 9 is set)	Device Reset Active
0x0204	0x009	**0x0004** (bit 9 is cleared)	Device Reset Cleared
0x0004	0x108	**0x0104** (bit 8 is set)	Core Reset Active
0x0804	0x100	**0x0004** (bit 8 is cleared)	Core Reset Cleared

Enabling the xSPI-Controller after Reset

When SLL xSPI-MBMC is configured in static mode, the xSPI-Controller mode is automatically set to **xSPI Host Controller Enabled**. No further actions are needed.

When SLL xSPI-MBMC is configured in generic mode, on reset the xSPI-Controller mode is automatically set to **No Core Enabled**.

If the user access the AXI/AVMM memory port during this time, an error on the AXI/AVMM is generated and the **Dec_error** (bit 1) flag in the **xSPI-MC_IRQ Status** register is set. The **CtrlNotActive** in the **xSPI-MC_Decode Status** register is also set.

To switch the controller to active mode, the user needs to **write 0x102 to xSPI-MC_CTRL.**

Next step in the process is to clear the hardware device reset.

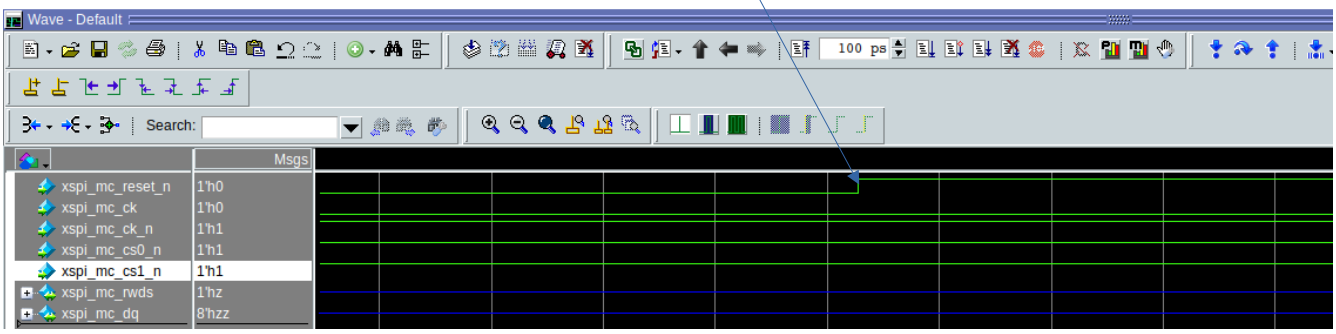
Controlling the Device hardware reset (*active low* reset going to PSRAM – Flash device)

When SLL xSPI-MBMC is configured in static mode, the device hardware reset is released when SLL xSPI-MBMC input reset (s00_axi_aresetn or i_iavs0_rstn) goes high. No further actions are needed.

When SLL xSPI-MBMC is configured in generic mode, the device hardware reset is still low when SLL xSPI-MBMC input reset (s00_axi_aresetn or i_iavs0_rstn) goes high. So we need to release it through software.

To do this, we write **0x009** to the **xSPI-MC_CTRL**. This will clear the dev_rst bit in the **xSPI-MC_CTRL** register**.** After this command, Device hardware reset goes high

The waveform below shows an example for releasing the device hardware reset after the FPGA powers up.



Next step in the process is to enable the xSPI memory controller for memory access.

Enabling the xSPI-Controller for Memory Access

All PSRAM/Flash devices have a power-up time period between the release of the hardware reset and the first memory access.

On reset, the **xSPI-MC_CTRL**/Setup ready bit is cleared.

When SLL xSPI-MBMC is configured in static mode and on the release of the reset, the xSPI-Controller starts a hardware controlled powering up timer. The value loaded in the timer is controlled via a hardware parameter. When the timer is ready, **xSPI-MC_CTRL**/**Setup_ready** is automatically set to 1 and the controller is ready for memory access.

The AXI/AVMM memory port is held in wait state if it is accessed during this time.

When SLL xSPI-MBMC is configured in generic mode, the user needs to set the **xSPI-MC_CTRL**/**Setup_ready** manually after ensuring that the power-up time period for the memory devices is met.

The user also needs to check that the hardware controlled powering up timer is ready. This is done by polling the **xSPI-MC Status register** and waiting until **PwrUp** (bit 16) is cleared.

If the user access the AXI/AVMM memory port when **xSPI-MC_CTRL**/**Setup_ready** is low, an error on the AXI/AVMM is generated and the **Dec_error** (bit 1) flag in the **xSPI-MC_IRQ Status** register is set. The **SetupNotReady** bit in the **xSPI-MC_Decode Status** register is also set.

To enable the controller for data access, the user needs to **write 0x10A to xSPI-MC_CTRL** This will set the **xSPI-MC_CTRL**/**Setup_ready** bit.

6.2.17 xSPI-MC_DLL_CTRL Register *(may not be used)*

bit	*Name*	*Description*	*Supported values*
3:0	DLL_cntrl_val	Delay control Value for DLL module	0 : 15
7:2	Reserved		

6.2.18 xSPI-MC_SPI-TO_DDR0/1 Register *(may not be used)*

bit	*Name*	*Description*	*Supported values*
63	en_spi_cmd	Enable SPI command	Should be set to 1
62	spi_cs	Memory Device Chip Select	0, 1
53:48	xspi_cmd_len	SPI Command length	Command length -1
47:0	spi cmd	Encoded Command to send to Memory Device. Bit 47 will be the first bit that is shifted out	

Some memory devices power up in SPI (x1) mode. SLL xSPI-MC contains a simple SPI controller that can send commands to the memory device in SPI (x1) mode. The command is defined by setting the XSPI_MC_SPI_TO_DDR0/1 registers. Always write XSPI_MC_SPI_TO_DDR0 register first.

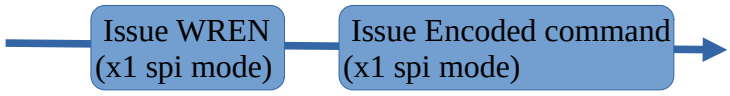
This feature can be used during initialization to switch the memory device from SPI (x1) mode to X8 DDR with Read Strobe so it can be used with SLL xSPI-MC controller. ** This feature is not used if the design has a QSPI controller. In this case, at power up, the QSPI controller sends SPI (x1) commands to the memory device before switching to SLL xSPI-MC controller.

When XSPI_MC_SPI_TO_DDR1 is written over th APB bus, this module will send a SPI command to the memory device.

Here is an example: XSPI_MC_SPI_TO_DDR0/1 = 64'h8027810000000E70

- Bit 64 = 1
- Bit 63 = 0 : Selects Chip Select 0
- Bits 53:48 = 0x27 ($39_{decimal}$) SPI Command length of 40
- Bits 47:8 = 0x81000000E7 bit encoded spi command
- bits 7:0 = 0 : Not used in this case since command length in 40 bits

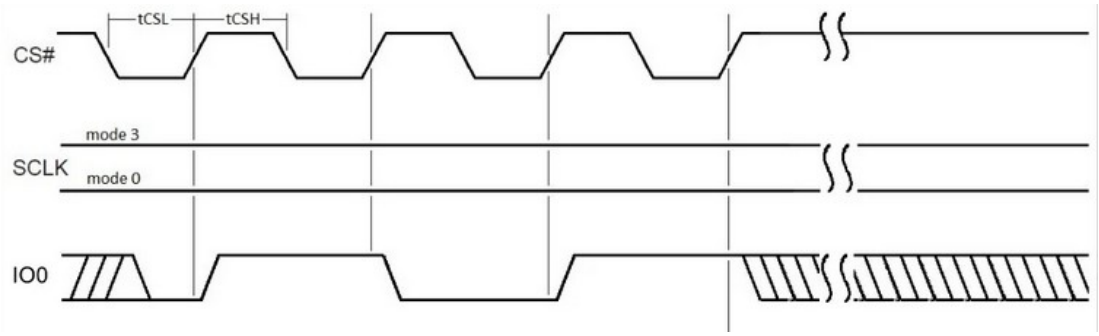
Please note that the controller issues a WREN (0x06) command before issuing the encoded command.



6.2.19 xSPI-MC_JEDEC251_Rst Register *(may not be used)*

bit	*Name*	*Description*	*Supported values*
31:15	Key	Key command to activate reset sequence	0, 0x5AA5
15:12	--	Not used	0
11:8	Thold_count	Hold time divider	0-15
7:4	cs_div	ChipSelect High/Low divider	0-15
3	--	Not used	0, 1
~~2~~	~~ck_polarity~~	~~Polarity of CK pin~~	~~Not used~~
1	spi_cs	Memory Device Chip Select	0, 1
0	rst_en	When reset_en==1 and key==0x5A5A, Jedec251 reset is issued to the memory device	0, 1

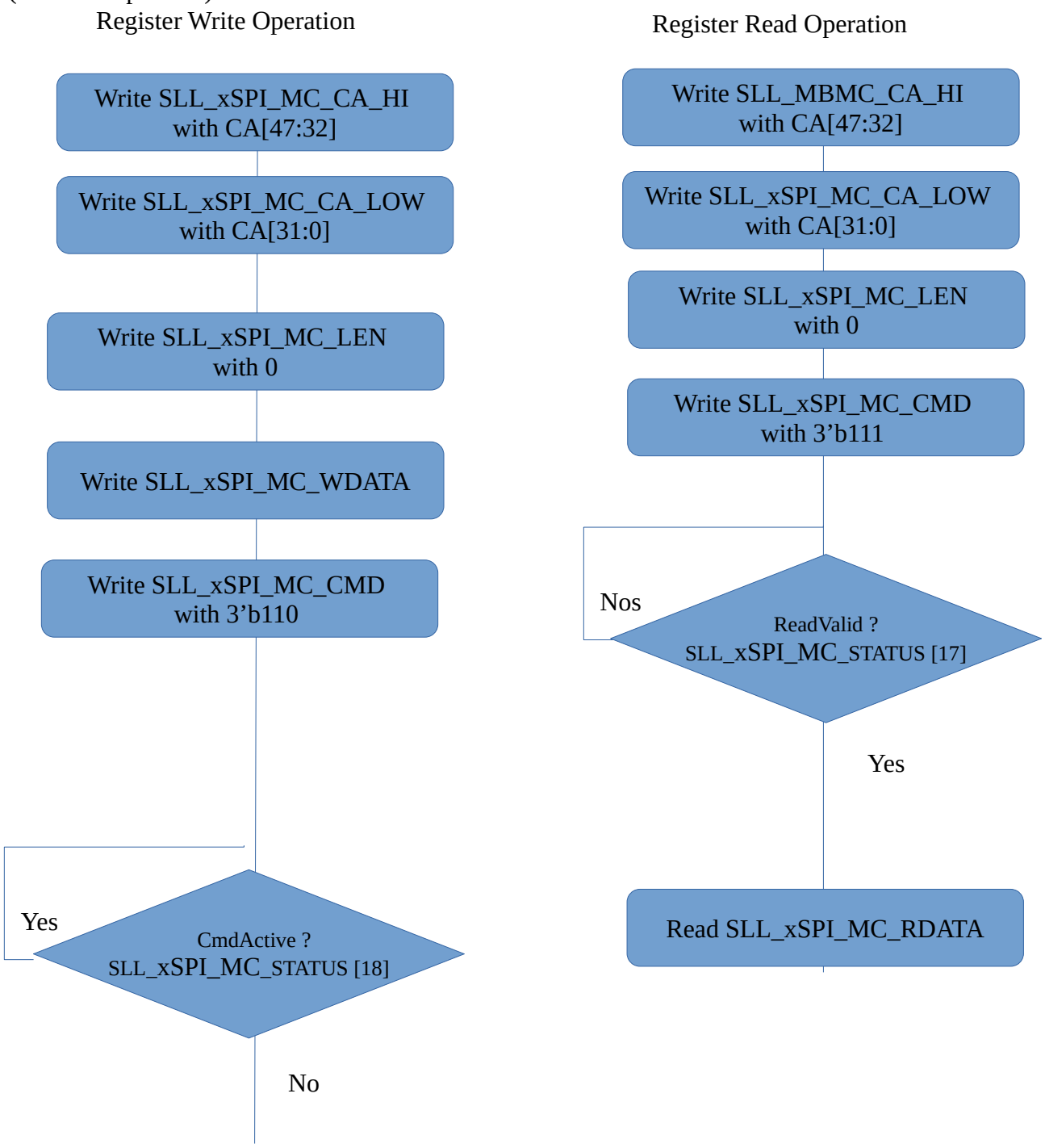
The JEDEC RESET (JESD252) command sequence can also be used to reset the device to a state similar to the power-on state without cycling power. The reset sequence does not use the CK pin. The CK pin is held low through the entire reset sequence. The Jedec reset command consists of sending pattern 0101 on DQ[0] pin for four consecutive positive edges of the CS pin.



When XSPI_MC_JEDEC251_Rst register is written over th APB bus with key == 0x5A5A and rst_en==1, this module will send a JEDEC 251 reset sequence to the memory device.

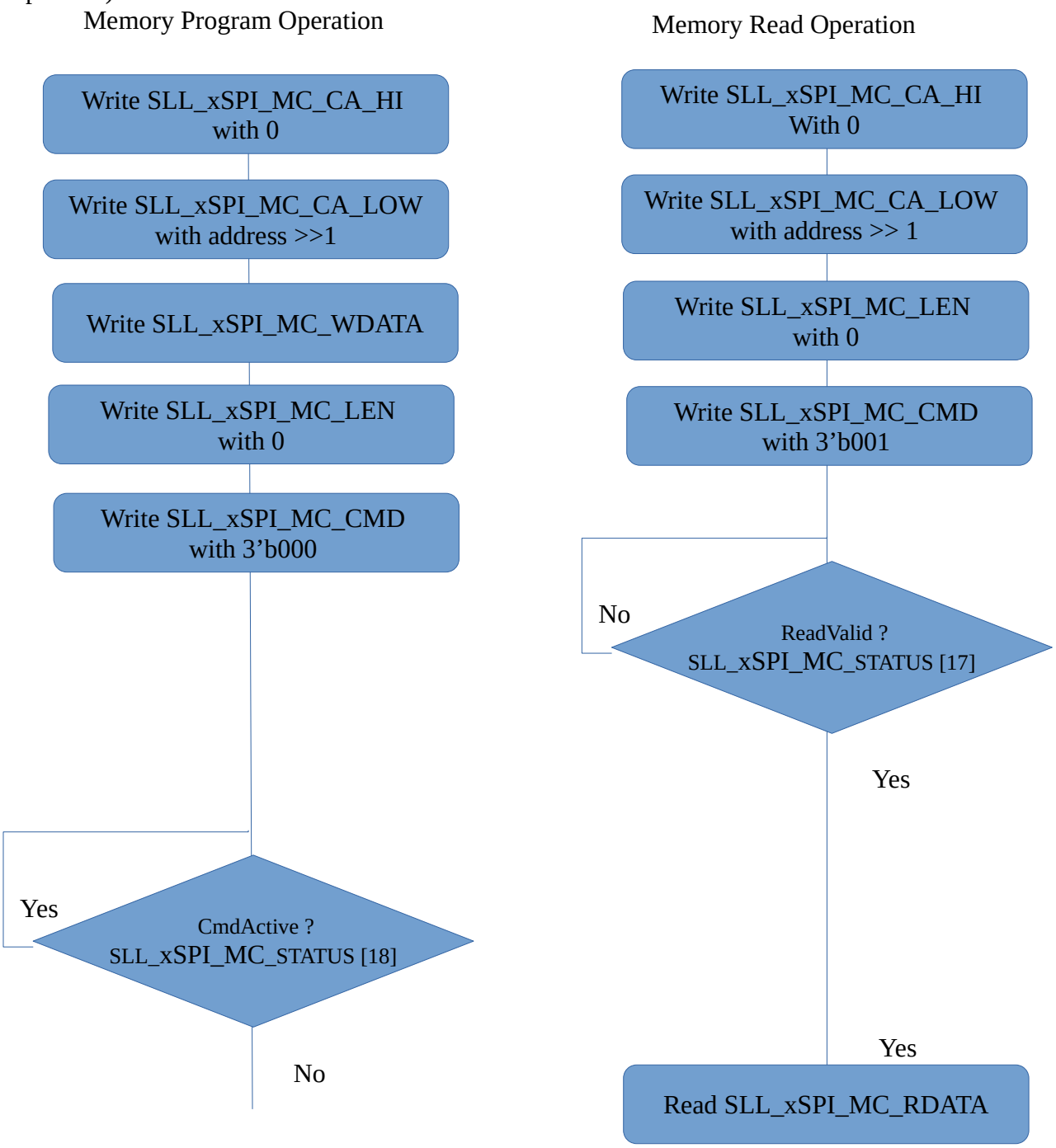
6.3 Accessing PsuedoRAM Registers

The follow flowchart shows how to use the control port interface to access the PseudoRAM registers. In this example, it is assumed that the PseudoRAM is connected to chipselect 1 (second chip select).



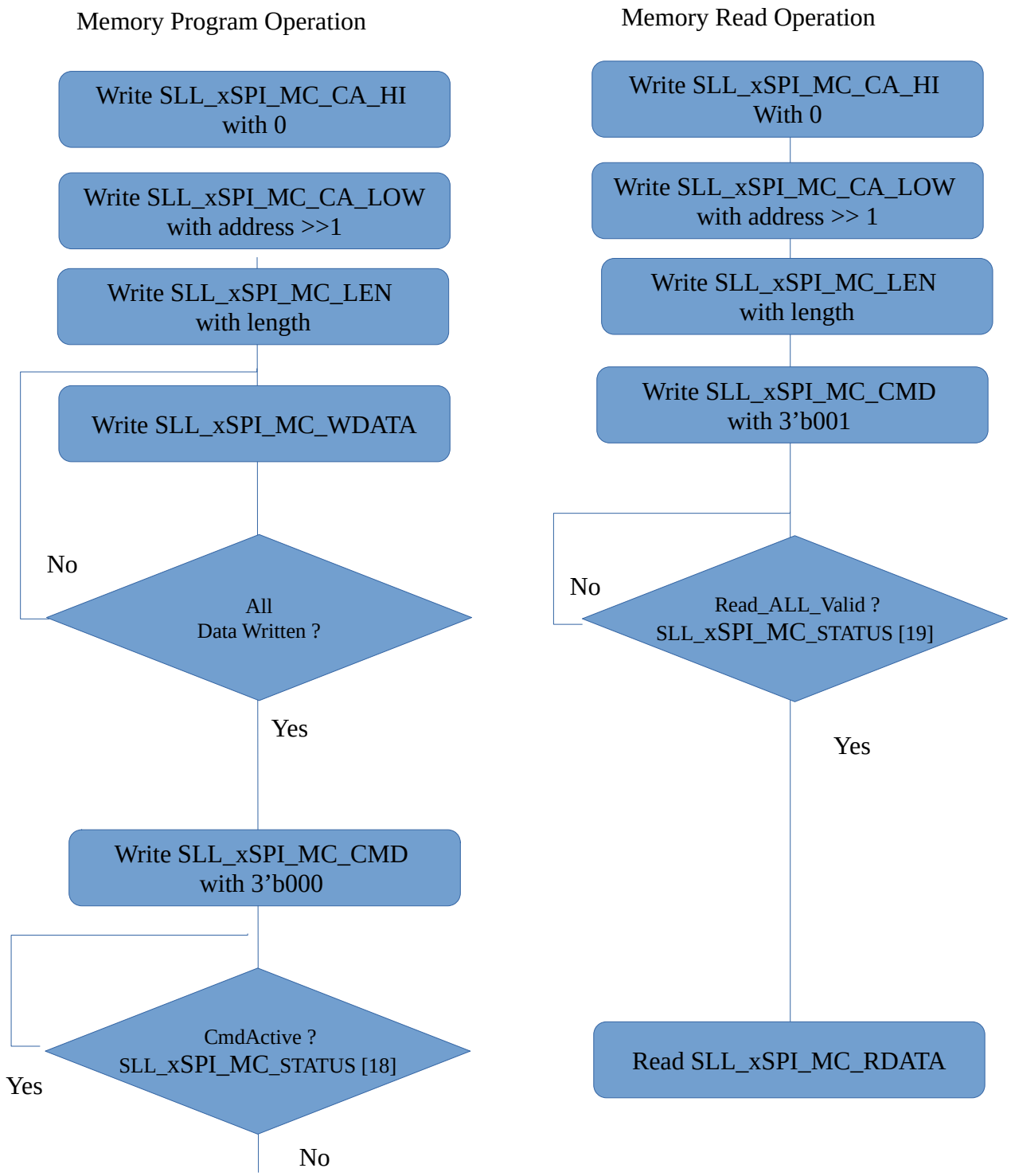
6.4 Accessing HyperFlash Memory using the Control Port

The follow flowchart shows how to use the control port interface to access the HyperFlash memory. In this example, it is assumed that the HyperFlash is connected to chipselect 0 (first chip select).



6.5 Accessing xSPI Flash Memory using the Control Port

The follow flowchart shows how to use the control port interface to access the xSPI memory. In this example, it is assumed that the xSPI Flash is connected to chipselect 0 (first chip select).

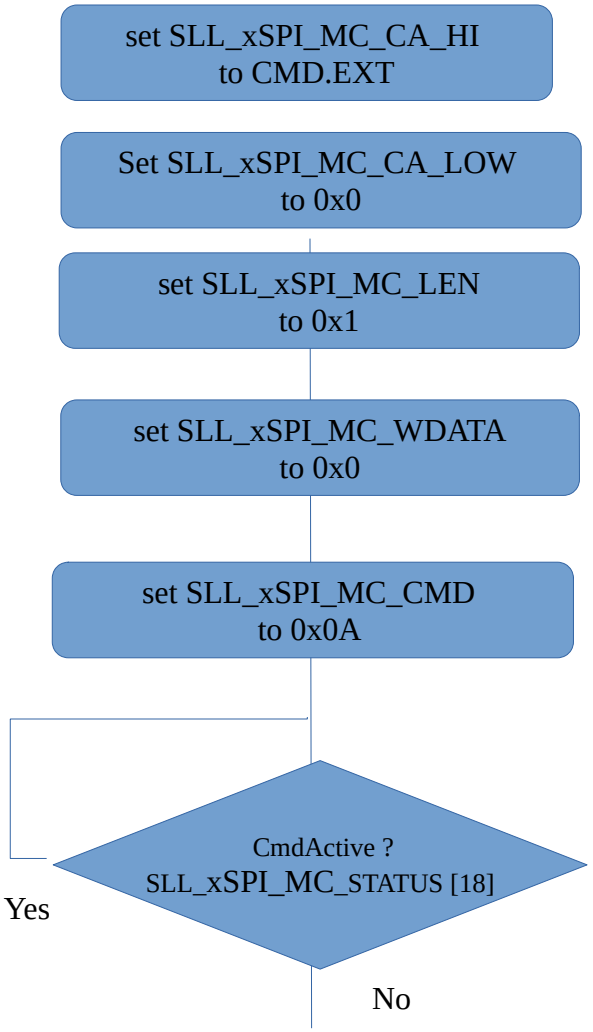


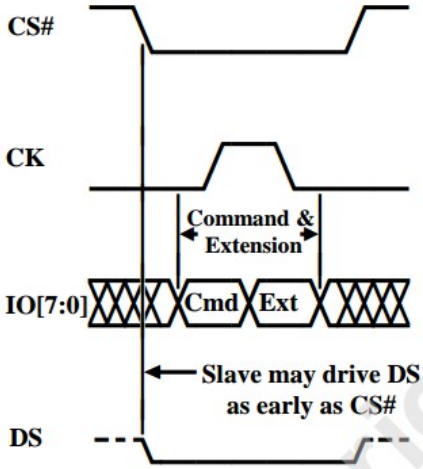
6.6 xSPI Profile 1.0 Transaction : Format 1.A

The follow flowchart shows how to use the control port interface to issue a transaction in format 1.A used with xSPI Profile 1.0 memory devices. In this example, it is assumed that the xSPI Flash is connected to chipselect 0 (first chip select).

xSPI-MC CMD register								
CA_CTL [19:18]	CA_CK [17:16]	LAT [15:8]	SPI [7]	Size [6]	CmdAddrOnly [5:3]	CS [2]	RegSpace [1]	Read [0]
2’b00	2’b00	8’b0	1’b0	1’b0	3’b001	cs	1’b1	1’b0

Format 1.A Transaction



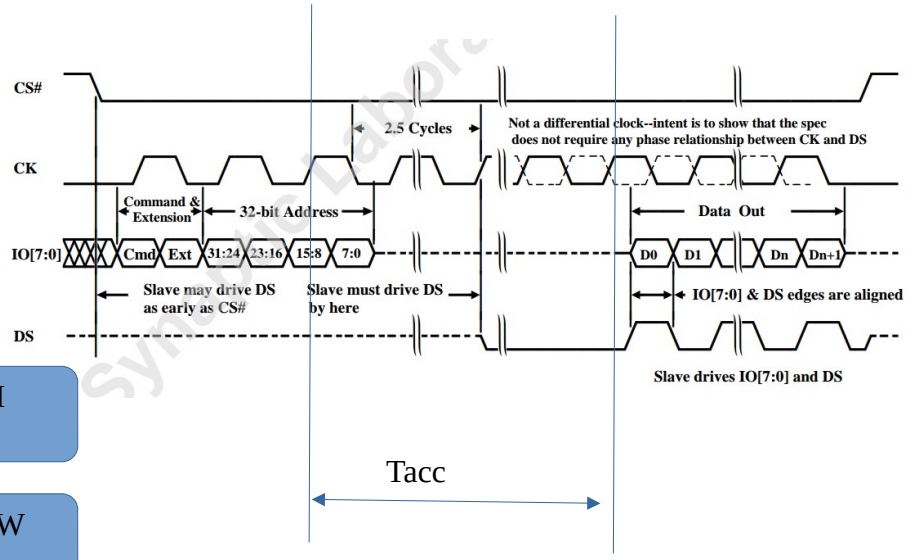


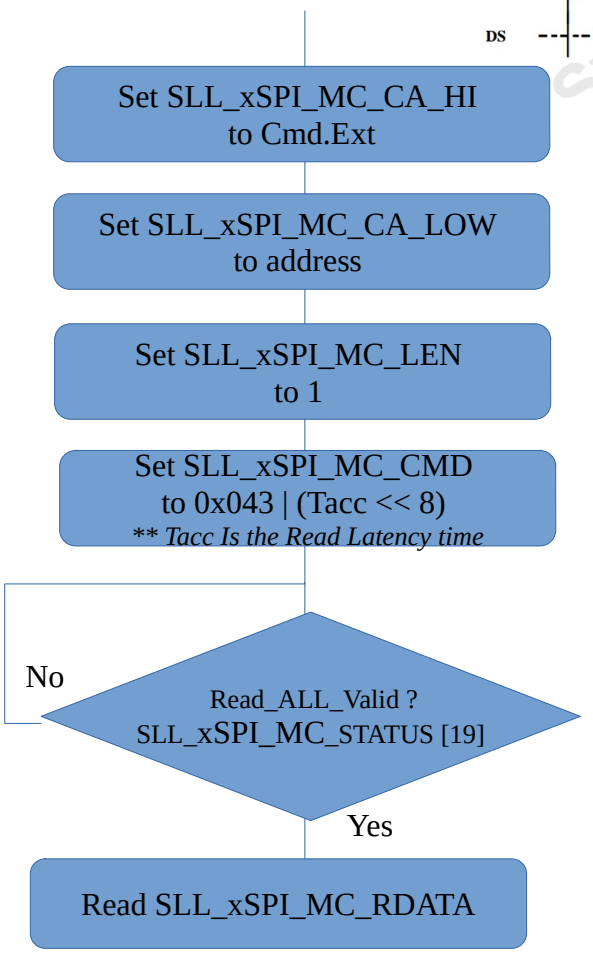
6.8 xSPI Profile 1.0 Transaction : Format 1.B

The follow flowchart shows how to use the control port interface to issue a transaction in format 1.B used with xSPI Profile 1.0 or xSPI Profile 2.0 memory. In this example, it is assumed that the xSPI Flash is connected to chipselect 0 (first chip select) .

xSPI-MC CMD register								
CA_CTL [19:18]	CA_CK [17:16]	LAT [15:8]	SPI [7]	Size [6]	CmdAddrOnly [5:3]	CS [2]	RegSpace [1]	Read [0]
2'b00	2'b00	Tacc	1'b0	1'b1	3'b000	cs	1'b1	1'b1

Format 1.B transaction Operation



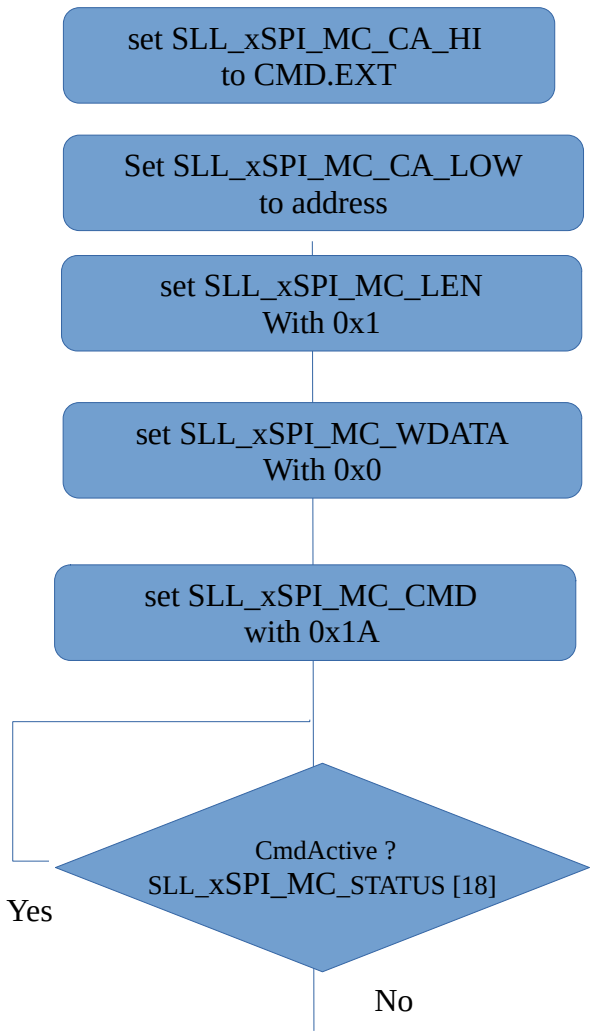


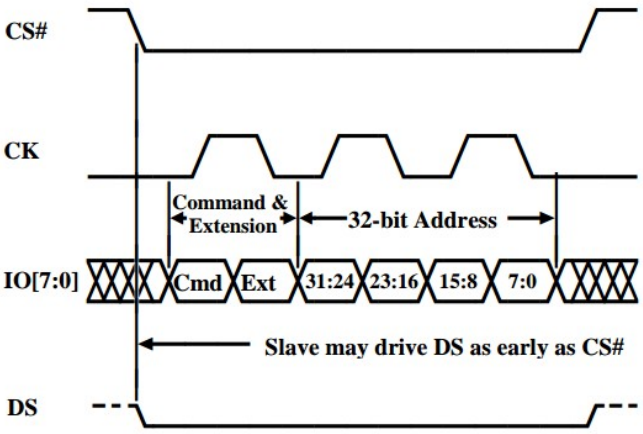
6.9 xSPI Profile 1.0 Transaction : Format 1.C

The follow flowchart shows how to use the control port interface to issue a transaction in format 1.C used with xSPI Profile 1.0 memory. In this example, it is assumed that the xSPI Flash is connected to chipselect 0 (first chip select).

xSPI-MC CMD register								
CA_CTL [19:18]	CA_CK [17:16]	LAT [15:8]	SPI [7]	Size [6]	CmdAddrOnly [5:3]	CS [2]	RegSpace [1]	Read [0]
2’b00	2’b00	8’b0	1’b0	1’b0	3’b011	cs	1’b1	1’b0

Format 1.C Transaction



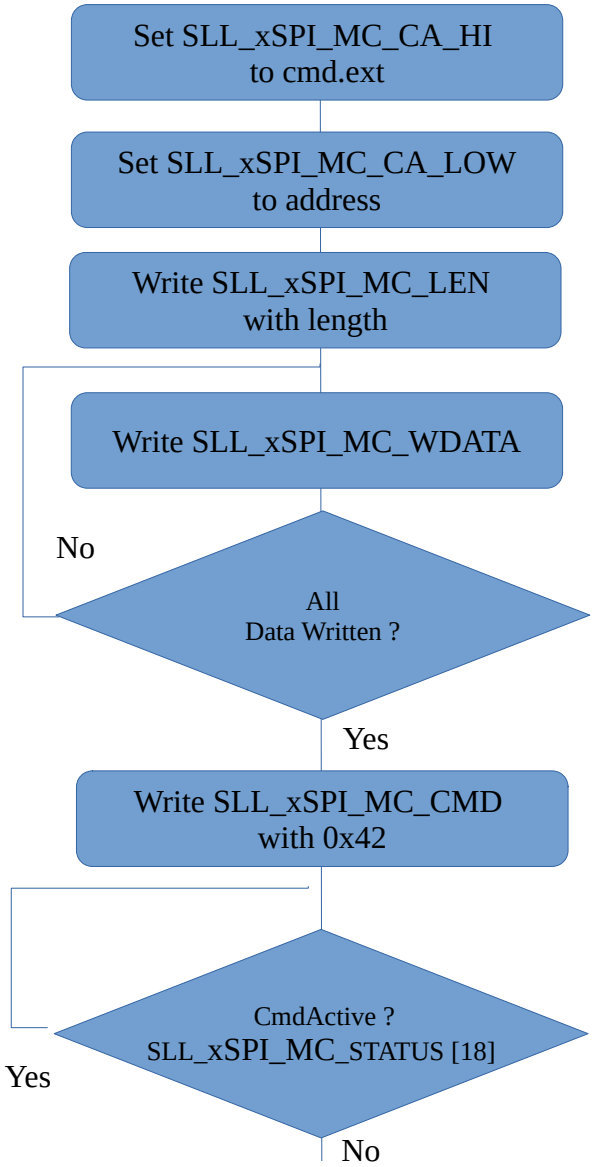


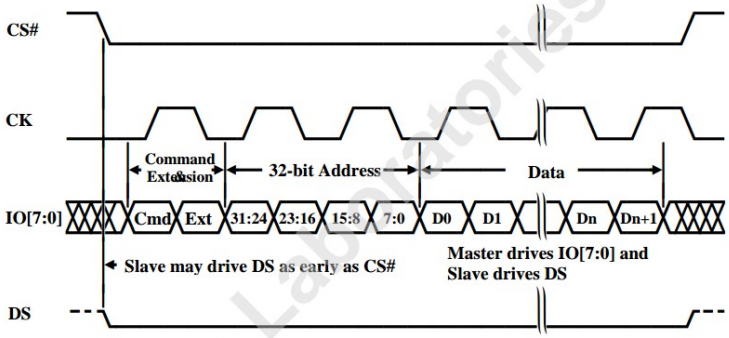
6.10 xSPI Profile 1.0 Transaction : Format 1.D

The follow flowchart shows how to use the control port interface to issue a transaction in format 1.D used with xSPI Profile 1.0 memory. In this example, it is assumed that the xSPI Flash is connected to chipselect 0 (first chip select). If only 16-bits are to be transferred, Size is set to 0, otherwise, size = 1.

xSPI-MC CMD register								
CA_CTL [19:18]	CA_CK [17:16]	LAT [15:8]	SPI [7]	Size [6]	CmdAddrOnly [5:3]	CS [2]	RegSpace [1]	Read [0]
2'b00	2'b00	8'b0	1'b0	Size (0/1)	3'b000	cs	1'b1	1'b0

Memory Program Operation





6.11 SLL Low Level drivers

Software flash/psram low lever drivers and a sample test application are provided on request. To date, low level drivers are provides for Cypress/Infineon Semper Flash with octospi (28HS/28HL series), Cypress/Infineon Hyper Flash (26KS/26KLand 26HS/26HL series) Micron xSPI Flash (MT35XU series), ISSI (IS25LX series), GigaDevice (GD55LX-GD25LX series) , Macronix Octal Flash (MX25 series) and Everspin STT-MRAM (EM064LX). Kindly contact Synaptic Labs' if you want additional flash drivers or functionality / API for a specific memory device.

Source code **sll_sll_ca_xspi_mc_t004_reg.c/h** provides basic access to SLL xSPI-MBMC IP via the APB Control Port.
You probably won't use this library directly; it's intended by be used as the hardware interface for a library that provides easier access to NOR Flash access and programming operations.
For example, xSPI-MBMC Initialisation drivers (**sll_ca_xspi_mc_init.c/h**) and NOR Flash Low level drivers (**sll_drvr_nor_flash.c/h**) uses this source code.
Structure **xSPI-MBMC_Info** needs to be instantiated for each instance of SLL xSPI-MBMC. It will hold basic information about the particular instance of SLL xSPI-MBMC IP such as Control Port base address (csr_base), Memory Device parameters connected on Chip Select 0 and Memory Device parameters connected on Chip Select 1.

```
//-----------------------------------------------
/* Controller Parameters   */
//-----------------------------------------------
typedef struct{
    uint32_t csr_base;
    uint32_t mem_freq_mhz;

    /*Device 0 parameters*/
    uint32_t dev0_name;
    uint32_t dev0_mem_base;
    uint32_t dev0_mem_size;

    /*Device 1 parameters*/
    uint32_t dev1_name;
    uint32_t dev1_mem_base;
    uint32_t dev1_mem_size;
} xSPI_MBMC_Info;
```

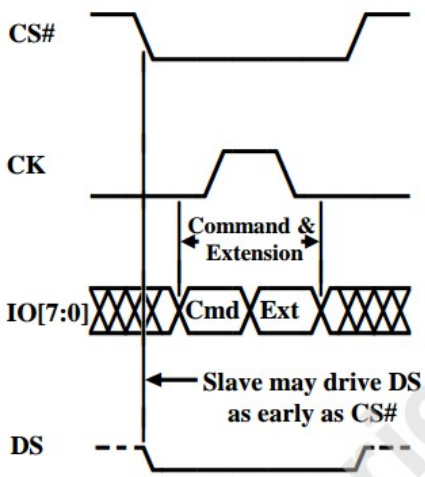
xSPI Profile 1.0 Transaction : Format 1.A

The follow command should be sent to the control port interface to issue a transaction in format 1.A used with xSPI Profile 1.0 memory devices :

sll_cs0_xspi_p1_format_1a_write(b,cad_high)

sll_cs1_xspi_p1_format_1a_write(b,cad_high)

- b is the base csr address of the control port and
- cad_high is the 16-bit CMD_EXT data to be issued on the bus.

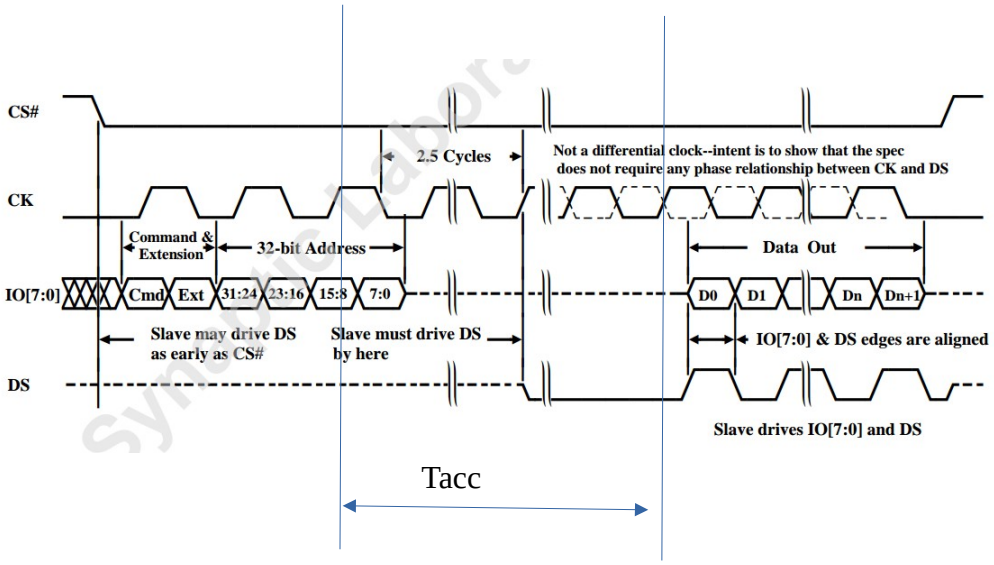


xSPI Profile 1.0 Transaction : Format 1.B

The follow command should be sent to the control port interface to issue a transaction in format 1.B used with xSPI Profile 1.0 or xSPI Profile 2.0 memory devices :

sll_xspi_mc_reg_read (b, cad_high, cad_low, cmd)

- b is the base csr address of the control port and
- cad_high is the 16-bit CMD_EXT data to be issued on the bus.
- cad_low is the 32-bit address to be issued on the bus.



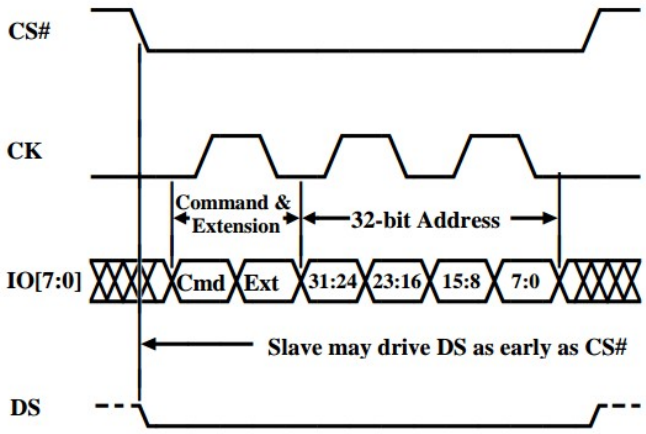
xSPI Profile 1.0 Transaction : Format 1.C

The follow command should be sent to the control port interface to issue a transaction in format 1.C used with xSPI Profile 1.0 memory devices :

sll_cs0_xspi_p1_format_1c_write(b,cad_high, cad_low)

sll_cs1_xspi_p1_format_1c_write(b,cad_high, cad_low)

- b is the base csr address of the control port and
- cad_high is the 16-bit CMD_EXT data to be issued on the bus.
- cad_low is the 32-bit address to be issued on the bus.



xSPI Profile 1.0 Transaction : Format 1.D

The follow command should be sent to the control port interface to issue a transaction in format 1.D used with xSPI Profile 1.0 memory devices :

To Write 16-bit values :

sll_cs0_xspi_p1_format_1d_write_x16(b,cad_high, cad_low, data_x16)

sll_cs1_xspi_p1_format_1d_write_x16(b,cad_high, cad_low, data_x16)

To Write 32-bit values :

sll_cs0_xspi_p1_format_1d_write(b,cad_high, cad_low, data_x32)

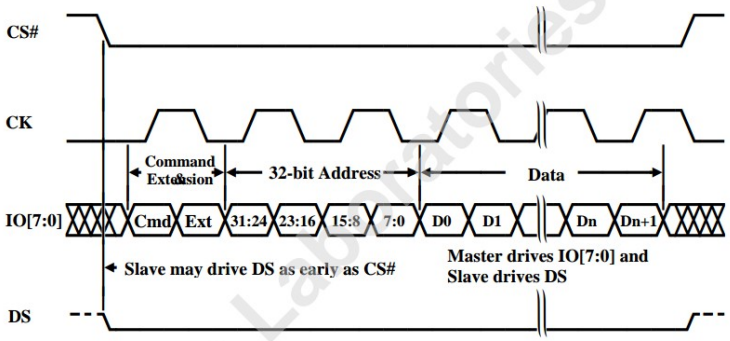
sll_cs1_xspi_p1_format_1d_write(b,cad_high, cad_low, data_x32)

To Write multiple 32-bit values :

sll_cs0_xspi_p1_format_1d_write_blk(b,cad_high, cad_low, count, wbuffer)

sll_cs1_xspi_p1_format_1d_write_blk(b,cad_high, cad_low, count, wbuffer)

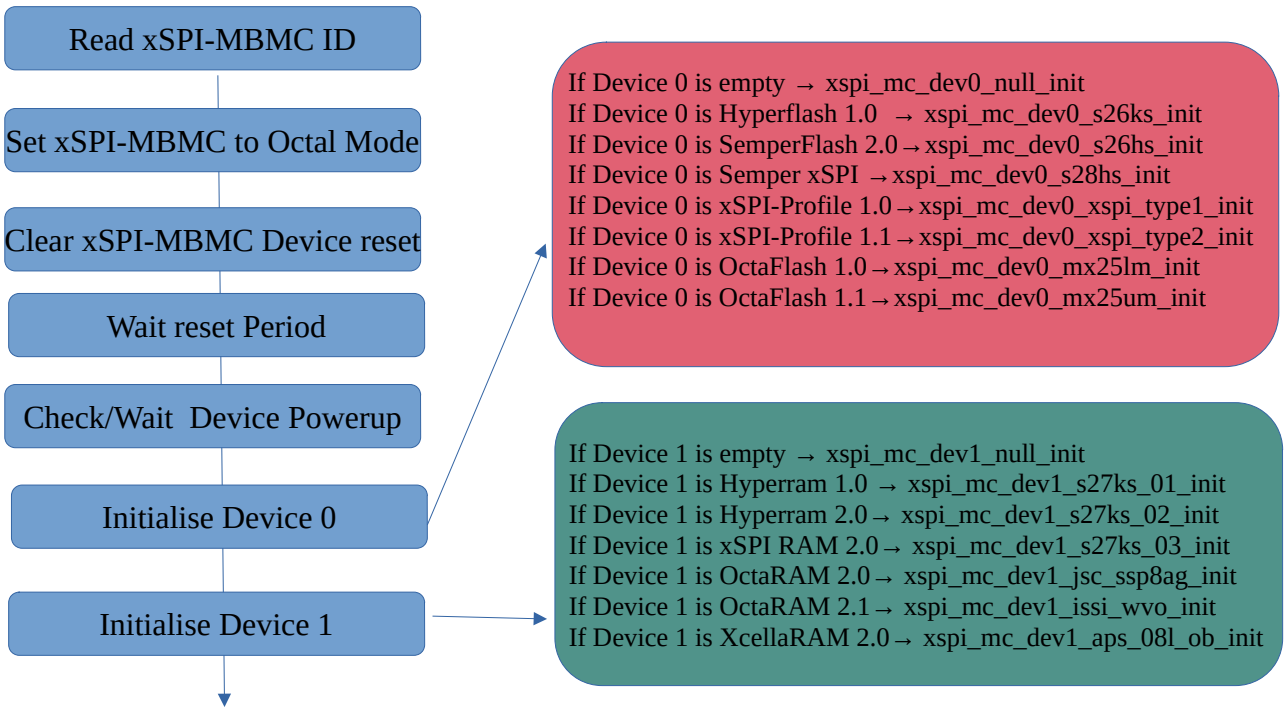
- b is the base csr address of the control port and
- cad_high is the 16-bit CMD_EXT data to be issued on the bus.
- cad_low is the 32-bit address to be issued on the bus.
- Count is the number of bytes to be issued
- data_x16, data_x32 or wbuffer is the data to be issued on the bus



6.12 SLL Initialisation drivers

Source code **sll_ca_xspi_mc_init.c/h** provides basic API for initialising SLL xSPI-MBMC IP according to the memory device(s) present on your system.

In particular, task **xspi_mc_cntrl_init** is used to initialise the SLL xSPI-MBMC controller. The following steps are performed:



6.13 SLL Flash drivers

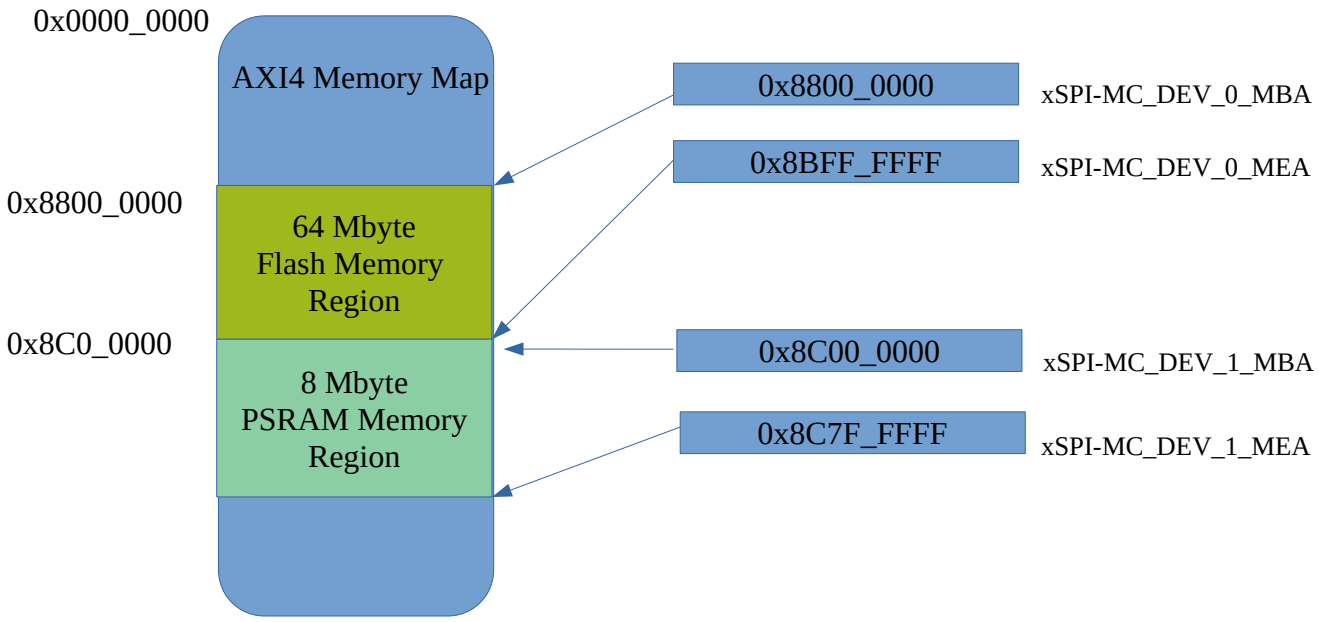
Source code **sll_drvr_nor_flash.c/h** provides basic access to NOR Flash memories. It supports :

- Flash Sofware Reset (sll_nor_flash_drvr_ResetCmd)
- Flash device ID query (sll_nor_flash_drvr_GetDeviceId)
- Flash flag status query (sll_nor_flash_drvr_GetFlagStatus)
- Flash status query (sll_nor_flash_drvr_GetStatus)
- Flash Read any Register (sll_nor_flash_drvr_ReadAnyReg)
- Flash Read any Non Volatile Register (sll_nor_flash_drvr_ReadAnyNVReg)
- Flash Poll Status register (sll_nor_flash_drvr_Poll)
- Flash Read CFI (sll_nor_flash_drvr_ReadCFI) ****where applicable*
- Flash Write any Register (sll_nor_flash_drvr_WriteAnyReg)
- Flash Write any Register with 16-bit value (sll_nor_flash_drvr_WriteAnyReg_x16)
- Flash Write any Non Volatile Register with 16-bit value (sll_nor_flash_drvr_ReadAnyNVReg_x16)
- Flash Write Data to Buffer (sll_nor_flash_drvr_WriteBufferProgramOp)
- Flash Write 32-bit to Single Location (sll_nor_flash_drvr_ProgramOp)
- Flash 4K Sector erase (sll_nor_flash_drvr_Sector4kEraseOp)
- Flash Sector erase (sll_nor_flash_drvr_SectorEraseOp) ****erase size depends on sector size)*
- Flash Bulk Memory Program Operation (sll_nor_flash_drvr_memcpy)

Please refer to the memory device(s) data sheets for further information about how the actual memory device should be accessed/programmed.

7.0 AXI4 Memory Address Mapping

SLL xSPI-MC controller has a single AXI/AVMM channel interface. For each memory device present on the system, the user needs to create a memory partition. When the controller is programmed in static mode, the values in memory base address (MBA) registers are automatically generated by the FPGA GUI configurator. When the controller is programmed in generic mode, the memory base address (MBA) registers need to be set by the user. This is done by programming the memory base address control registers available in the register area though the AVMM/APB register port.



The example below shows a memory partitioning example for a 64 Mbte HyperFlash and an 8Mbyte HyperRam connected to the system. The first 64 Mbyte memory region is reserved for the HyperFlash memory. The second 8 Mbyte memory region is reserved for the HyperRam memory.

In this example, the base addresses for the HyperFlash and HyperRAM is calculated as follows :

- HyperFlash start address is : 0x8800_0000
- HyperFlash end address is : 0x88FF_FFFF (0x8800_0000 + 0x03FF_FFFF)
- HyperRAM start address is : 0x8C00_0000

The memory base address control registers need to be updated as follows :

- Write xSPI-MC_DEV_0_MBA with 0x8800_0000
- Write xSPI-MC_DEV_1_ MBA with 0x8C00_0000

This controller can't assert two Chip Select at the same time.

The way memory devices are accessed is as follows :

CS0# select condition:

- xSPI-MC_DEV_0_MBA <= AxADDR < xSPI-MC_DEV_0_MBA + DEV0 Size
- Address for Main Controller = AxADDR - xSPI-MC_DEV_0_MBA

CS1# select condition:

- xSPI-MC_DEV_1_MBA <= AxADDR < xSPI-MC_DEV_1_MBA + DEV01 Size
- Address for Main Controller = AxADDR - xSPI-MC_DEV_1_MBA

Example RiscV linker script

This linker script needs to be edited according to the memory devices present in the design. The start address and size of the memory space must be correct as per your design.
The example below shows 3 (arbitrary) memory regions :

- a 64K byte onchip SRAM located at base address 0x80000000
- a 64M byte HyperFlash located at base address 0x88000000
- an 8M byte HyperRAM located at base address 0x8C000000

```
MEMORY
{
   ram (rwx) : ORIGIN = 0x80000000, LENGTH = 64k
   hyperflash (rx) : ORIGIN = 0x88000000, LENGTH = 64M
   hyperram (rwx) : ORIGIN = 0x8C000000, LENGTH = 8M
}
```

8.0 xSPI Controller Core Operation

The xSPI Controller Core is accessed through the

- AXI/AVMM Read Channel bus
- AXI/AVMM Write Channel bus
- Internal Memory access generated through APB Register Control Port.

Memory access over the AXI/AVMM Read And Write Channels always have precedence over the Internal Memory access generated through APB/AVMM Register Control Port. The AXI/AVMM Read and Write Channels have the same access priority and a round robin arbiter controls which operation is taken.

- If there is a concurrent Read and Write request, the round robin arbiter controls which operation takes place.
- If there is no read request and there is some write requests, the write request passes through the arbiter.
- If there is no write request and there is some read requests, the read request passes through the arbiter.
- If there is an Internal Memory request generated through APB/AVMM Register Control Port, the xSPI Controller will wait until the AXI/AVMM Read and Write channel bus are free.

8.1 AXI Memory Response Error (*for AXI-inerfaces only*)

The AXI protocol provides response signaling for both read and write transactions:

- for read transactions the response information from the slave is signaled on the read data channel. This is signaled by RRESP.
- for write transactions the response information is signaled on the write response channel. This is signaled by BRESP.

RRESP and BRESP encoding

RRESP[1:0] BRESP[1:0]	Response
0b00	OKAY
0b01	EXOKAY
0b10	SLVERR
0b11	DECERR

The following is the behaviour of the RRESP and BRESP signals

- OKAY response during normal operation.
- DECERR response when AXI tries to access an address that is not mapped in the address space or the address accesses more than one memory region
- SLVERR response when there is a FIFO Error, the core is not enabled, there is a timeout error on the read path or the controller is in power-down mode

- SLVERR response is also triggered on a read response while ECC_2_bit error is active on the RX FIFO RAM (when implemented). ECC_1_bit errors on the RX FIFO RAM do not throw SLVERR response since they are silently corrected.

9.0 Core Reset

9.1 Global Asynchronous Reset

This section describes requirement of **s00_axi_aresetn** signal on the AXI and APB interface. When low, **s00_axi_aresetn** signal will reset

- AXI Core logic
- Memory Controller logic
- Forces Registers in the APB region to their default value
- It will force the external xSPI RSTn signal

Reset Pulse width must be greater than > 30us *** check memory data-sheets for more info*

9.2 Software Core Reset *(not implemented)*

The Memory Controller Core can also be reset through software by setting the corresponding bit in the xSPI-MC_MODE_CTRL Register . In this case, only the memory controller core is reset. It is recommended that the Reset Pulse width is greater than > 300us.

9.3 Software Device Reset

The external memory devices can also be reset through software by setting the corresponding bit in the xSPI-MC_MODE_CTRL Register. When high, it will force the external xSPI RSTn signal to low.

It is recommended that the Reset Pulse width is greater than > 30us *** check respective memory data-sheets for more info*

9.3 Device Sleep/Power Down *(to be further defined)*

The external memory devices can be powered down or set to sleep mode to reduce power consumption. This is done by issuing memory specific register commands to the memory device. After issuing register commands, set the Sleep/PowerDown bit to 1.

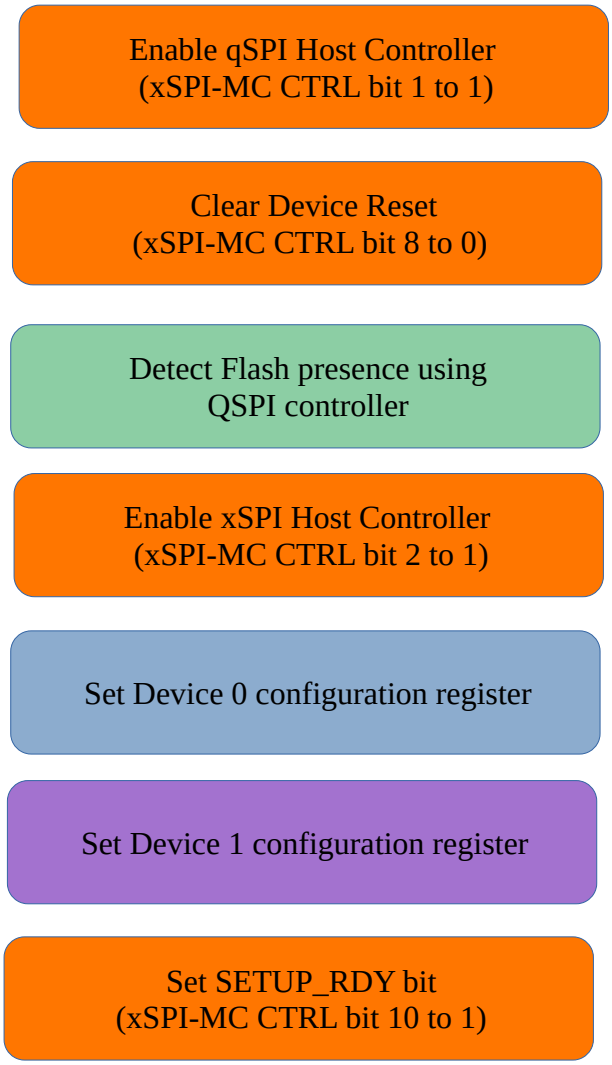
To signal the end of sleep/power down, set the Sleep/PowerDown bit needs to be 0.

When this bit changes from high to low, it will force the external xSPI CSn signal to low for a period of 1us to wake up the devices.

Alternatively, a Software Device Reset command can be used to wake up the device.

10.0 Powering Up *(generic mode only)*

The process of powering up xSPI-MC is as follows :



HyperRAM 1.0 and HyperRAM 2.0 Device Configuration ***(generic mode only)***

The process to configure a HyperRAM 1.0 and HyperRAM 2.0 device is as follows :

Set Memory Region using registers
xSPI-MC_DEV_x_MBA

Set Timing Control using register
xSPI-MC_DEV_x_TIM

Set Config Control using register
xSPI-MC_DEV_x_CFG

Set RW Commands using registers
xSPI-MC_DEV_x_CMD

HyperFlash/SemperFlash with Hyperbus Interface Device Configuration ***(generic mode only)***

The process to configure a HyperFlash and SemperFlash with Hyperbus Interface Device is as follows :

Set Memory Region using registers
xSPI-MC_DEV_x_MBA

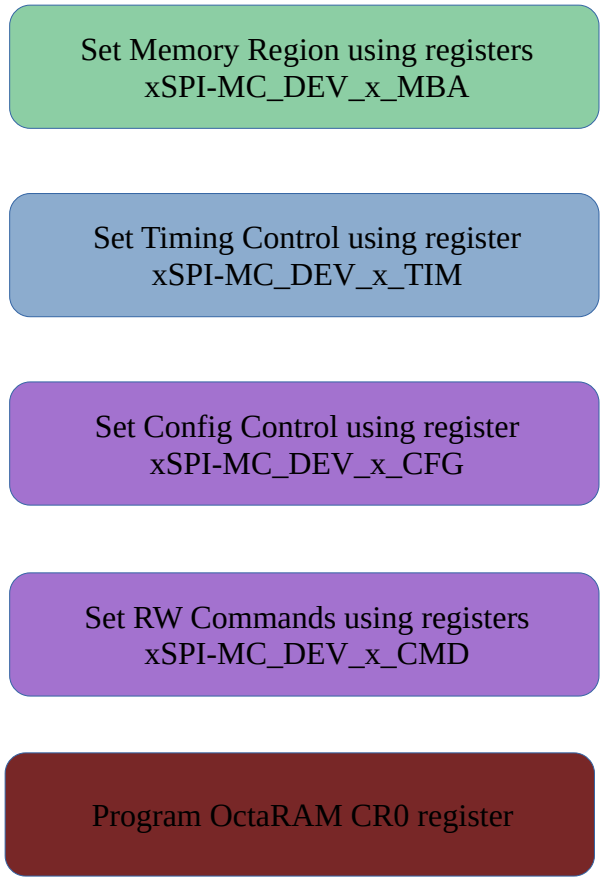
Set Timing Control using register
xSPI-MC_DEV_x_TIM

Set Config Control using register
xSPI-MC_DEV_x_CFG

Set RW Commands using registers
xSPI-MC_DEV_x_CMD

OctaRAM Device Configuration ***(generic mode only)***

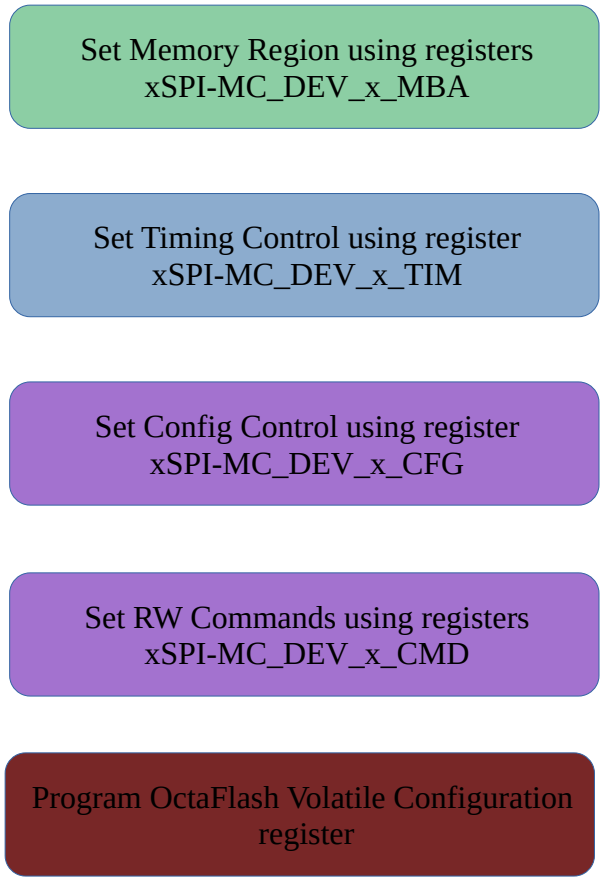
The process to configure an OctaRAM memory device is as follows :



** Please note the extra step needed for programming the OctaRAM CR0 register.

OctaFlash Device Configuration ***(generic mode only)***

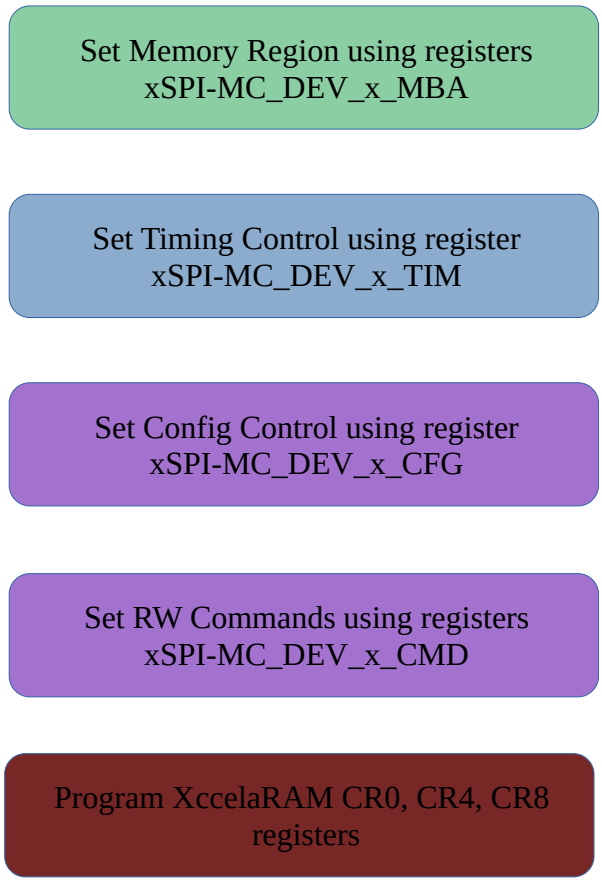
The process to configure a OctaFlash memory device is as follows :



** Please note the extra step needed for programming the OctaFlash Volatile Configuration register.

XccelaRAM Device Configuration *(generic mode only)*

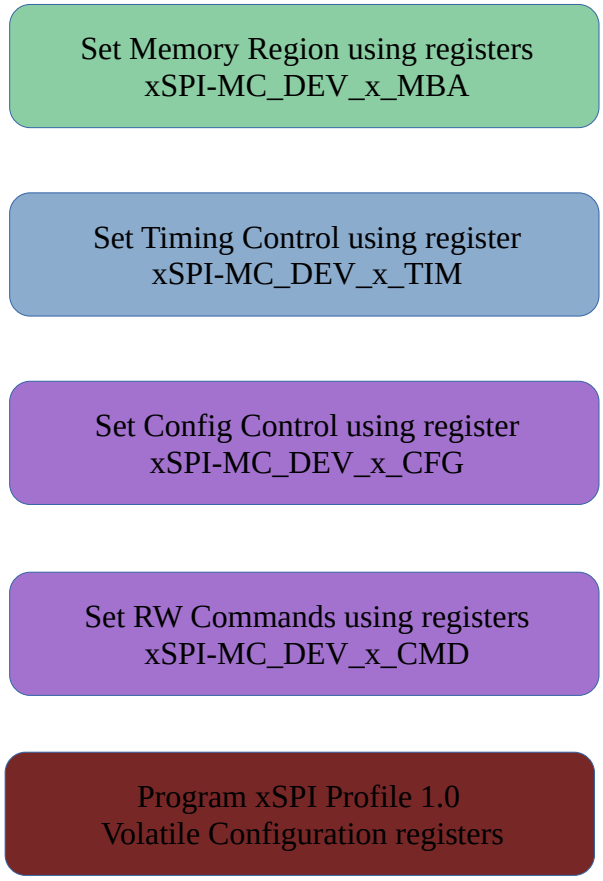
The process to configure a XcellaRAM memory device is as follows :



** Please note the extra step needed for programming the XccelaRAM CR0,CR4 and CR8 registers.

xSPI Profile 1.0 Flash Device Configuration *(generic mode only)*

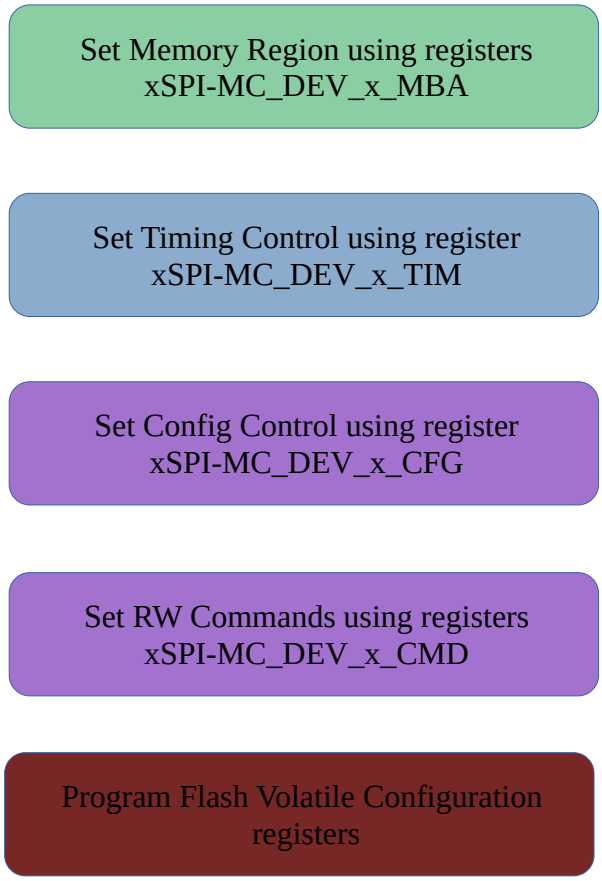
The process to configure the xSPI Profile 1.0 memory device is as follows :



** Please note the extra step needed for programming the xSPI Profile 1.0 volatile configuration registers.

xSPI Profile 1.0 Flash Device Configuration *(generic mode only)*

The process to configure the xSPI Profile 1.0 memory device is as follows :



** Please note the extra step needed for programming the appropriate Flash Volatile configuration registers.

11.0 CycloneV Support

Targeting Higher clock speeds on Intel's Cyclone V (and Cyclone V with HPS) FPGA family.

On the Cyclone V, there are two different ways to interface with PSRAM/Flash enabled memories:

- Using a General Purpose I/O (GPIO) pin library

  This technique works on all Intel FPGA device families. However, the top clock speed achievable for accessing DDRx memories using GPIO libraries varies depending on the choice of pins, and on the FPGA device family. The GPIO library is not well suited to driving SDRAM on Cyclone V families. Maximum speed is around 50 Mhz

- Using Intel's integrated DDR/IO registers.

  These registers integrated in the I/O pads permit considerably higher clock speeds over the GPIO direct implementation on the same FPGA device. The new version of SLL xSPI-MC IP (v3.3.x upward) now provide support this configuration.

Additional QSF constraint

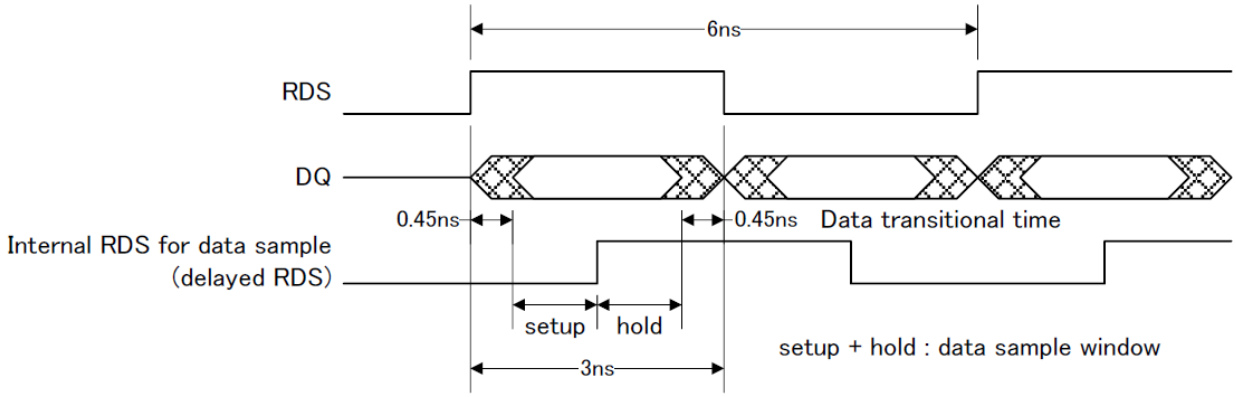
** Very important : The user needs to add the following constraint in the top level QSF file.

```
set_global_assignment -name VERILOG_MACRO "CYCLONE_V_DDIO=1"
```

Kindly ask for a reference design in case static timing is not achieved.

12.0 Static Timing Consideration

12.1 Data Input Timing Constraint



The input read strobe (MB_RWDS) signal is edge aligned to the data signal (HB_dq).

12.2 Timing Constraints

A timing constraint script (sll_ca_xspi_mc_top.sdc) will be automatically generated by Synaptic Labs' xSPI memory Controller Qsys component. This script will be placed in Quartus Prime Synthesis directory and executed automatically during the Quartus Prime compilation process.

12.3 Pin Clustering

Please ensure all PSRAM/Flash channel pins are clustered physically close together in the programmable FPGA fabric. The FPGA board designer will need to balance the place-and-route requirements of the Multi-Bus memory Controller Logic against the ideal placement of pins from the board layout perspective to minimize skew across pins and to minimise pin-to-pin wire latency delay.

The signals received on the MB_DQ pins are fed as an input into a single on-chip SRAM (e.g. M9K). The parallel capture of those signals is clocked by MB_RWDS. Hence the location of the MB_DQ/MB_RWDS pins must be placed in a way to also ensure low wire latencies to that single on-chip SRAM. From the perspective of the PSRAM/Flash memory controller, try to ensure that the data being transported over all DQ signals arrive as close as possible, with as little skew, at the I/O pads”

The following output signals { MB_CLK0, MB_CLK0n, MB_CLK1, MB_CLK1n}

- use an **altera_gpio_lite/altera_ddio** output pad configured in DDR register mode.

The following bi-direction signals MB_Dq and MB_RWDS

- (Output mode) - use an **altera_gpio_lite/altera_ddio** output pad with output enable control (oe) and configured in DDR register mode.
- (Input mode) - Unregistered buffer mode (pass through) or **altera_gpio_lite/altera_ddio** input pad in DDR register mode

Please ensure these DDR signals are mapped to IO Elements with DDR capabilities.

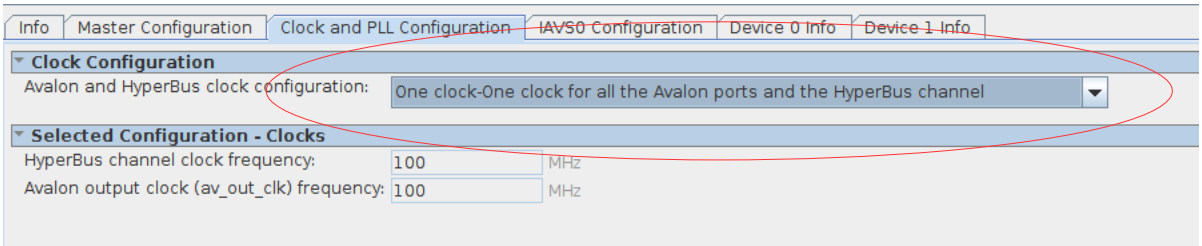
13.0 Using 3V PSRAM and HyperFlash devices

13.1 Selecting the correct operating frequency in Qsys

The 3V PSRAM/HyperRAM and Flash memories supports a lower frequency than the 1.8V memory devices. Hence the designer needs to select a frequency that is supported by the memory device.

In Qsys, open SLL MBMC IP component. Locate the Clock and PLL configuration Tab. Ensure that **The memory channel clock Frequency** is 100Mhz or less.

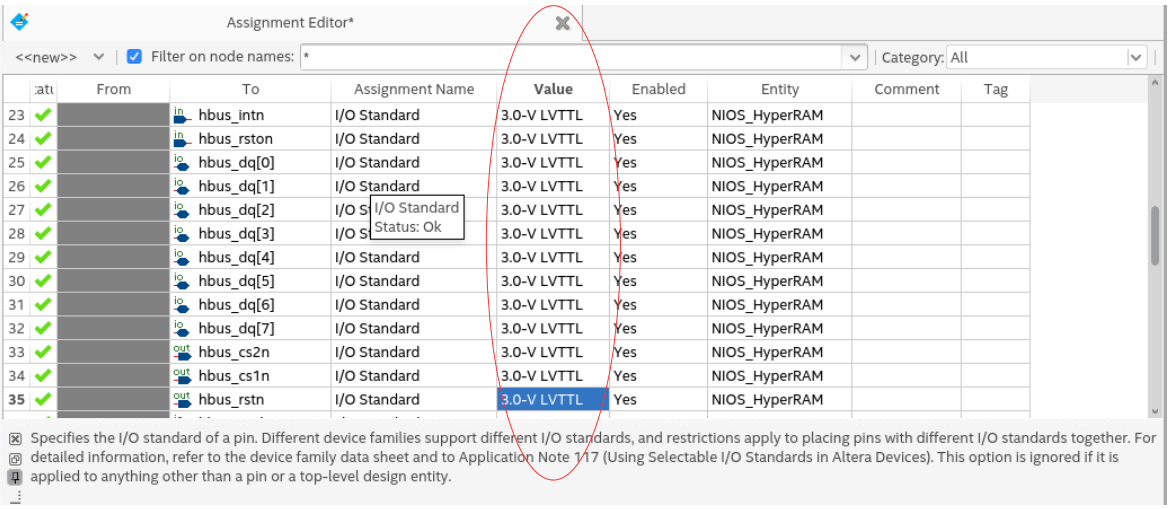
We suggest that the designer use the One Clock operation for both the Avalon Port and the PSRAM/Flash Channel Port.



13.2 Selecting the correct voltage in Quartus

The designer needs to set the PSRAM/Flash voltage level signals to 3V.

In Quartus, open the Assignment editor (Quartus→ Assignment → Assignment Editor). Set the PSRAM/Flash memory signals voltage level to 3V.



13.3 Connecting the PSRAM signals to the FPGA I/O

Some PSRAM and Flash devices do NOT require a differential clock pair. HB_CLK0n is left unconnected.

The figure below shows a typical top level connection for a 3V HyperRAM only design. Note that some signals are left unconnected since they are not used in the design.

```
-------------------------------------
-- Instantiation of main QSys system
-------------------------------------
u0_main : component lab2
   port map (
      clk_clk       => c10_clk50m,
      reset_reset_n => c10_resetn,

      sll_hyperbus_HB_RSTn  => hbus_rstn,    --  .HB_RSTn
      sll_hyperbus_HB_CLK0  => hbus_clk0p,   --  .HB_CLK0
      sll_hyperbus_HB_CLK0n => open          --  .HB_CLK0n
      sll_hyperbus_HB_CLK1  => open,         --  .HB_CLK1
      sll_hyperbus_HB_CLK1n => open,         --  .HB_CLK1n
      sll_hyperbus_HB_CS0n  => open,         --  .HB_CS0n
      sll_hyperbus_HB_CS1n  => hbus_cs2n,    --  .HB_CS1n
      sll_hyperbus_HB_WPn   => open,         --  .HB_WPn
      sll_hyperbus_HB_RWDS  => hbus_rwds,    --  .HB_RWDS
      sll_hyperbus_HB_dq    => hbus_dq,      --  .HB_dq
      sll_hyperbus_HB_RSTOn => '1',          --  .HB_RSTOn
      sll_hyperbus_HB_INTn  => '1'           --  .HB_INTn

      pio_led_export => LEDs_debug,
   );
```

Additional QSF constraint

Sometimes the user needs to manually add delay constraints in the top level QSF file for the Read Strobe signal (RWDS).

```
set_instance_assignment -name PAD_TO_CORE_DELAY 2 -to hbus_rwds
```

Kindly ask for a reference design in case static timing is not achieved.